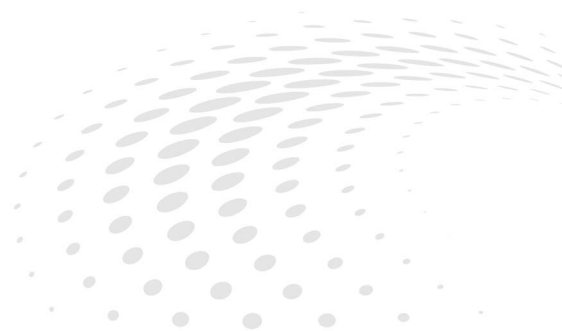


TUNING GUIDE

AMD EPYC 9004



Ubuntu®

(Versions 20.04.5 and 22.04)

Publication	58005
Revision	1.3
Issue Date	June, 2023

© 2023 Advanced Micro Devices, Inc. All rights reserved.

The information contained herein is for informational purposes only and is subject to change without notice. While every precaution has been taken in the preparation of this document, it may contain technical inaccuracies, omissions and typographical errors, and AMD is under no obligation to update or otherwise correct this information. Advanced Micro Devices, Inc. makes no representations or warranties with respect to the accuracy or completeness of the contents of this document, and assumes no liability of any kind, including the implied warranties of noninfringement, merchantability or fitness for particular purposes, with respect to the operation or use of AMD hardware, software or other products described herein. No license, including implied or arising by estoppel, to any intellectual property rights is granted by this document. Terms and limitations applicable to the purchase or use of AMD's products are as set forth in a signed agreement between the parties or in AMD's Standard Terms and Conditions of Sale.

Trademarks

AMD, the AMD Arrow logo, AMD EPYC, 3D V-Cache, and combinations thereof are trademarks of Advanced Micro Devices, Inc. Ubuntu is a registered trademark of Canonical, Ltd. PCIe is a registered trademark of PCI-SIG Corporation. Other product names and links to external sites used in this publication are for identification purposes only and may be trademarks of their respective companies.

* Links to third party sites are provided for convenience and unless explicitly stated, AMD is not responsible for the contents of such linked sites and no endorsement is implied.

Date	Version	Changes
July, 2022	0.1	Initial NDA partner release
Sep, 2022	0.2	Updated NDA partner release with additional command details
Nov, 2022	1.0	Initial public release
Dec, 2022	1.1	Minor errata corrections
Mar, 2023	1.2	Added 97xx OPN and AMD 3D V-Cache™ technology information
Jun, 2023	1.3	Second public release

Audience

This document is intended for a technical audience such as Ubuntu® application architects, production deployment, and performance engineering teams with a server configuration background who have:

- Admin access to the server's management interface (BMC).
- Familiarity with the server's management interface.
- Admin OS access.
- Familiarity with the OS-specific configuration, monitoring, and troubleshooting tools.

Authors

Canonical: Christian Eberhardt - **AMD:** Anthony Hernandez and Kim Naru

Note: All of the settings described in this Tuning Guide apply to all AMD EPYC 9004 Series Processors of all core counts with or without AMD 3D V-Cache™ except where explicitly noted otherwise.

Table of Contents

Chapter 1	Introduction	1
Chapter 2	AMD EPYC™ 9004 Series Processors	3
2.1	General Specifications	3
2.2	Model-Specific Features	3
2.3	Operating Systems	4
2.4	Processor Layout	4
2.5	“Zen 4” Core	4
2.6	Core Complex (CCX)	5
2.7	Core Complex Dies (CCDs)	5
2.8	AMD 3D V-Cache™ Technology	6
2.9	I/O Die (Infinity Fabric™)	7
2.10	Memory and I/O	8
2.11	Visualizing AMD EPYC 9004 Series Processors (Family 19h)	9
2.11.1	Models 91xx-96xx (“Genoa”)	9
2.11.2	Models 97xx (“Bergamo”)	10
2.12	NUMA Topology	10
2.12.1	NUMA Settings	10
2.13	Dual-Socket Configurations	12
Chapter 3	BIOS Defaults Summary	13
3.1	Processor Core Settings	14
3.2	Power Efficiency Settings	16
3.3	NUMA and Memory Settings	17
3.4	Infinity Fabric Settings	18
3.5	PCIe, I/O, Security, and Virtualization Settings	19
3.6	Higher-Level Settings	20
Chapter 4	Common Linux Kernel Tools and Examples	21
4.1	lscpu	22
4.2	numactl	23
4.3	numastat	24
4.4	lstopo / hwloc-ls	24
4.4.1	hwloc-info	26
4.5	cpupower	26
4.5.1	cpupower monitor	27
4.6	CPU Governors	27
4.7	htop	28
4.7.1	Example 1: Per-CPU Utilization Statistics	29
4.7.2	Example 2: Per-NUMA-Node Utilization Statistics	29

Chapter 5	General Tuning Recommendations	31
5.1	LLC as NUMA Domain	31
5.2	AMD uProf	32
5.3	perf	32
5.3.1	perf list cpu	33
5.3.2	perf list cache	34
5.3.3	perf stat	34
Chapter 6	Virtualization	37
6.1	Secure Encrypted Virtualization (SEV)	37
6.1.1	SEV Prerequisites	38
6.2	AMD EPYC Virtualization Support	38
6.3	Evaluating Workloads and VM Workloads	39
Chapter 7	Troubleshooting and Debugging Notes	41
7.1	Error Detection and Correction (EDAC)	41
7.1.1	Get the Memory Controller MCx Device Information	42
7.2	Error Injection using mce-inject	43
7.2.1	Installation	43
7.2.2	Usage	43
Chapter 8	Resources	45
Chapter 9	Processor Identification	47
9.1	CPUID Instruction	47
9.2	New Software-Visible Features	48
9.2.1	AVX-512	48

Chapter**1**

Introduction

This tuning guide describes parameters that can optimize performance of servers powered by AMD EPYC™ 9004 Series Processors running Ubuntu® versions 20.04.5 and 22.04.



This page intentionally left blank.

Chapter

2

AMD EPYC™ 9004 Series Processors

AMD EPYC™ 9004 Series Processors represent the fourth generation of AMD EPYC server-class processors. This generation of AMD EPYC processors feature AMD's latest "Zen 4" based compute cores, next-generation Infinity Fabric, next-generation memory & I/O technology, and use the new SP5 socket/packaging.

2.1 General Specifications

AMD EPYC 9004 Series Processors offer a variety of configurations with varying numbers of cores, Thermal Design Points (TDPs), frequencies, cache sizes, etc. that complement AMD's existing server portfolio with further improvements to performance, power efficiency, and value. Table 1-1 lists the features common to all AMD EPYC 9004 Series Processors.

Common Features of all AMD EPYC 9004 Series Processors	
Compute cores	Zen4-based
Core process technology	5nm
Maximum cores per Core Complex (CCX)	8
Max memory per socket	6 TB
Max # of memory channels	12 DDR5
Max memory speed	4800 MT/s DDR5
Max lanes Compute eXpress Links	64 lanes CXL 1.1+
Max lanes Peripheral Component Interconnect	128 lanes PCIe® Gen 5

Table 2-1: Common features of all AMD EPYC 9004 Series Processors

2.2 Model-Specific Features

Different models of 4th Gen AMD EPYC processors have different feature sets, as shown in Table 1-2.

AMD EPYC 9004 Series Processor (Family 19h) Features by Model		
Codename	"Genoa"*	"Bergamo"*
Model #	91xx-96xx	97xx
Max number of Core Complex Dies (CCDs)	12	8
Number of Core Complexes (CCXs) per CCD	1	2
Max number of cores (threads)	96 (192)	128 (256)
Max L3 cache size (per CCX)	1,152 MB (96 MB)♦	256 MB (16 MB)
Max Processor Frequency	4.4 GHz♦♦	3.15 GHz
Includes ♦AMD 3D V-Cache (9xx4X) and ♦♦high-frequency (9xx4F) models.		
*GD-122: The information contained herein is for informational purposes only and is subject to change without notice. Timelines, roadmaps, and/or product release dates shown herein and plans only and subject to change. "Genoa" and "Bergamo" are codenames for AMD architectures and are not product names.		

Table 2-2: AMD EPYC 9004 Series Processors features by model

2.3 Operating Systems

AMD recommends using the latest available targeted OS version and updates. Please see [AMD EPYC™ Processors Minimum Operating System \(OS\) Versions](#) for detailed OS version information.

2.4 Processor Layout

AMD EPYC 9004 Series Processors incorporate compute cores, memory controllers, I/O controllers, RAS (Reliability, Availability, and Serviceability), and security features into an integrated System on a Chip (SoC). The AMD EPYC 9004 Series Processor retains the proven Multi-Chip Module (MCM) Chiplet architecture of prior successful AMD EPYC processors while making further improvements to the SoC components.

The SoC includes the Core Complex Dies (CCDs), which contain Core Complexes (CCXs), which contain the “Zen 4”-based cores. The CCDs surround the central high-speed I/O Die (and interconnect via the Infinity Fabric). The following sections describe each of these components.

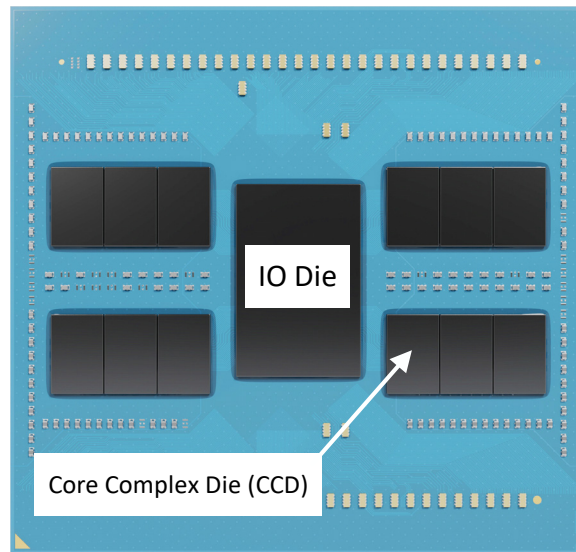


Figure 2-1: AMD EPYC 9004 configuration with 12 Core Complex Dies (CCD) surrounding a central I/O Die (IOD)

2.5 “Zen 4” Core

AMD EPYC 9004 Series Processors are based on the new “Zen 4” compute core. The “Zen 4” core is manufactured using a 5nm process and is designed to provide an Instructions per Cycle (IPC) uplift and frequency improvements over prior generation “Zen” cores. Each core has a larger L2 cache and improved cache effectiveness over the prior generation. Each “Zen 4” core includes:

- Up to 32 KB of 8-way L1 I-cache and 32 KB of 8-way of L1 D-cache
- Up to a 1 MB private unified (Instruction/Data) L2 cache.

Each core supports Simultaneous Multithreading (SMT), which allows 2 separate hardware threads to run independently, sharing the corresponding core’s L2 cache.

2.6 Core Complex (CCX)

Figure 2-2 shows a Core Complex (CCX) where up to eight “Zen 4”-based cores share a L3 or Last Level Cache (LLC). Enabling Simultaneous Multithreading (SMT) allows a single CCX to support up to 16 concurrent hardware threads.

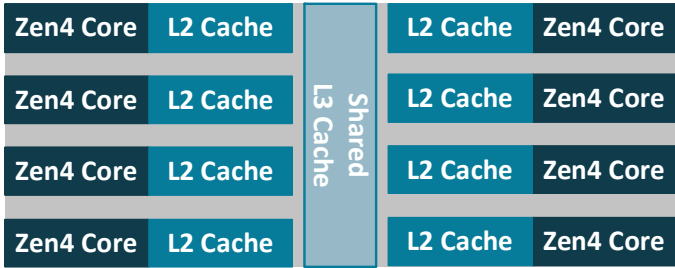


Figure 2-2: Top view of 8 compute cores sharing an L3 cache (91xx-96xx models)

2.7 Core Complex Dies (CCDs)

The Core Complex Die (CCD) in an AMD EPYC 9xx4 Series Processor may contain either one or two CCXs, depending on the processor (91xx-96xx “Genoa” vs. 97xx “Bergamo”), as shown in Figure 2-5.

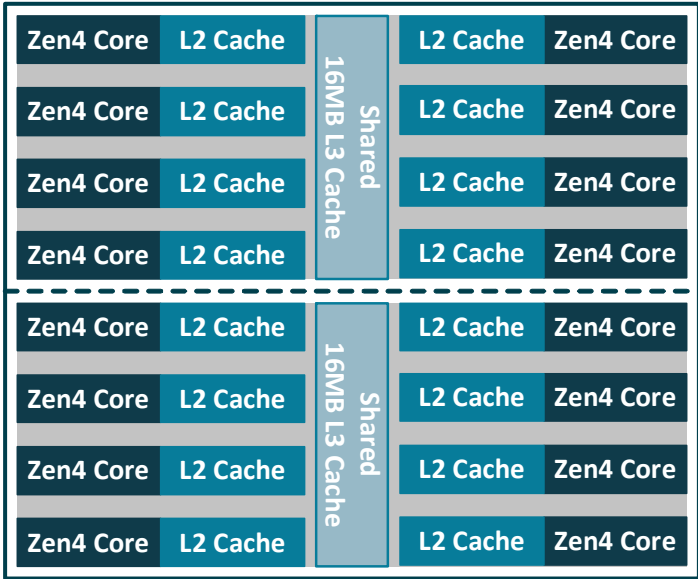


Figure 2-3: 2 CCXs in a single 4th Gen AMD EPYC 97xx CCD

Each of the Core Complex Dies (CCDs) in a 97xx model AMD EPYC 9004 Series Processor contains two CCXs (Figure 2-5):

AMD EPYC 9004 Series Processor	91xx-96xx	97xx
# of CCXs within a CCD	1	2

Table 2-3: CCXs per CCD by AMD EPYC model

You can disable cores in BIOS using one or both of the following approaches:

- Reduce the cores per L3 from 8 down to 7,6,5,4,3,2, or 1 while keeping the number of CCDs constant. This approach increases the effective cache per core ratio but reduces the number of cores sharing the cache.
- Reduce the number of active CCDs while keeping the cores per CCD constant. This approach maintains the advantages of cache sharing between the cores while maintaining the same cache per core ratio.

2.8 AMD 3D V-Cache™ Technology

AMD EPYC 9xx4X Series Processors include AMD 3D V-Cache™ die stacking technology that enables 97xx to achieve more efficient chiplet integration. AMD 3D Chiplet architecture stacks L3 cache tiles vertically to provide up to 96MB of L3 cache per die (and up to 1 GB L3 Cache per socket) while still providing socket compatibility with all AMD EPYC™ 9004 Series Processor models.

AMD EPYC 9004 Series Processors with AMD 3D V-Cache technology employ industry-leading logic stacking based on copper-to-copper hybrid bonding “bumpless” chip-on-wafer process to enable over 200X the interconnect densities of current 2D technologies (and over 15X the interconnect densities of other 3D technologies using solder bumps), which translates to lower latency, higher bandwidth, and greater power and thermal efficiencies.

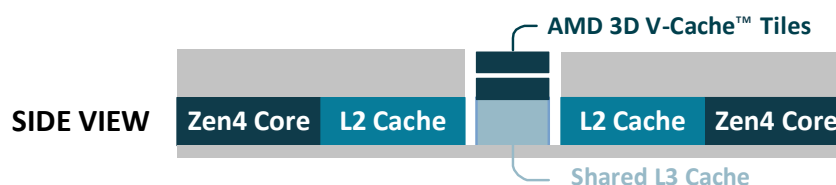


Figure 2-4: Side view of vertically-stacked central L3 SRAM tiles

AMD EPYC 9004 Series Processors	9xx4	9004X (with 3D V-Cache)
Max Shared L3 Cache per CCD	32 MB	96 MB

Table 2-4: L3 cache by processor model

Different OPNs also may have different numbers of cores within the CCX. However, for any given part, all CCXs will always contain the same number of cores.

2.9 I/O Die (Infinity Fabric™)

The CCDs connect to memory, I/O, and each other through an updated I/O Die (IOD). This central AMD Infinity Fabric™ provides the data path and control support to interconnect CCXs, memory, and I/O. Each CCD connects to the IOD via a dedicated high-speed Global Memory Interconnect (GMI) link. The IOD helps maintain cache coherency and additionally provides the interface to extend the data fabric to a potential second processor via its xGMI, or G-links. AMD EPYC 9004 Series Processors support up to 4 xGMI (or G-links) with speeds up to 32Gbps. The IOD exposes DDR5 memory channels, PCIe® Gen5, CXL 1.1+, and Infinity Fabric links.

All dies (chiplets) interconnect with each other via AMD Infinity Fabric technology. Figure 2-6 (which corresponds to Figure 2-2, above) shows the layout of a 96-core AMD EPYC 9654 processor. The AMD EPYC 9654 has 12 CCDs, with each CCD connecting to the IOD via its own GMI connection.

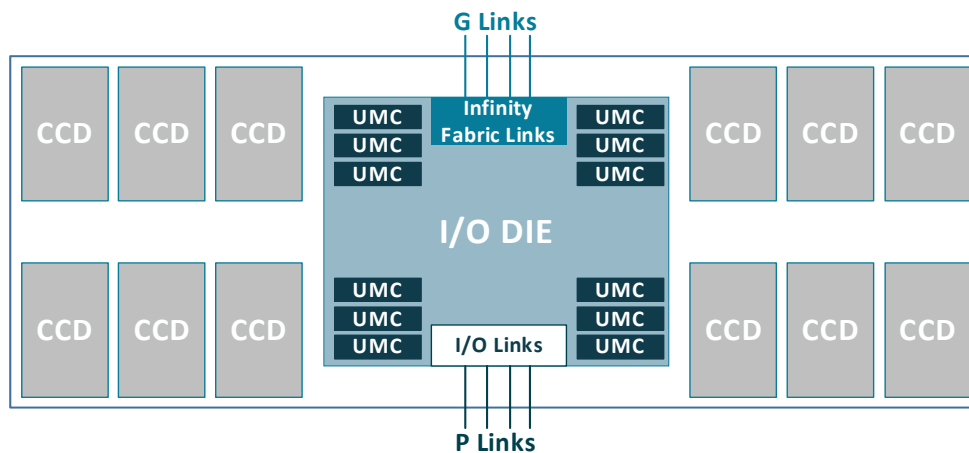


Figure 2-5: AMD EPYC 9654 processor internals interconnect via AMD Infinity Fabric (12 CCD processor shown)

AMD also provides “wide” OPNs (e.g. AMD EPYC 9334) where each CCD connects to two GMI3 interfaces, thereby allowing double the Core-to-I/O die bandwidth.

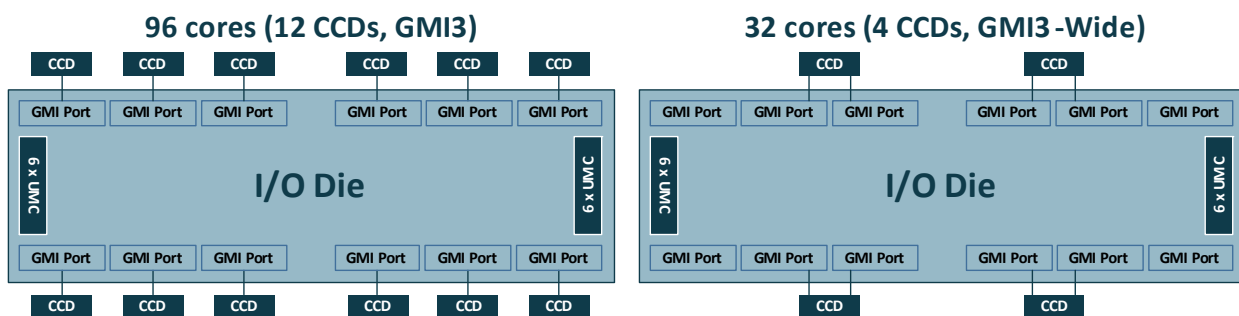


Figure 2-6: Standard vs. Wide GMI links

The IOD provides twelve Unified Memory Controllers (UMCs) that support DDR5 memory. The IOD also presents 4 ‘P-links’ that the system OEM/designer can configure to support various I/O interfaces, such as PCIe Gen5, and/or CXL 1.1+.

2.10 Memory and I/O

Each UMC can support up to 2 DIMMs per channel (DPC) for a maximum of 24 DIMMs per socket. OEM server configurations may allow either 1 DIMM per channel or 2 DIMMs per channel. 4th Gen AMD EPYC processors can support up to 6TB of DDR5 memory per socket. Having additional and faster memory channels compared to previous generations of AMD EPYC processors provides additional memory bandwidth to feed high-core-count processors. Memory interleaving on 2, 4, 6, 8, 10, and 12 channels helps optimize for a variety of workloads and memory configurations.

Each processor may have a set of 4 P-links and 4 G-links. An OEM motherboard design can use a G-link to either connect to a second 4th Gen AMD EPYC processor or to provide additional PCIe Gen5 lanes. 4th Gen AMD EPYC processors support up to eight sets of x16-bit I/O lanes, that is, 128 lanes of high-speed PCIe Gen5 in single-socket platforms and up to 160 lanes in dual-socket platforms. Further, OEMs may either configure 32 of these 128 lanes as SATA lanes and/or configure 64 lanes as CXL 1.1+. In summary, these links can support:

- Up to 4 G-links of AMD Infinity Fabric connectivity for 2P designs.
- Up to 8 x16 bit or 128 lanes of PCIe Gen 5 connectivity to peripherals in 1P designs (and up to 160 lanes in 2-socket designs).
- Up to 64 lanes (4 P-links) that can be dedicated to Compute Express Link (CXL) 1.1+ connectivity to extended memory.
- Up to 32 I/O lanes that can be configured as SATA disk controllers.

2.11 Visualizing AMD EPYC 9004 Series Processors (Family 19h)

This section depicts AMD EPYC 9004 Series Processors that have been set up with four nodes per socket (NPS=4). Please see [“NUMA Topology” on page 10](#) for more information about nodes.

2.11.1 Models 91xx-96xx (“Genoa”)

4th Gen AMD EPYC 9004 processors with model numbers 91xx-96xx have up to 12 CCDs that each contain a single CCX, as shown below.

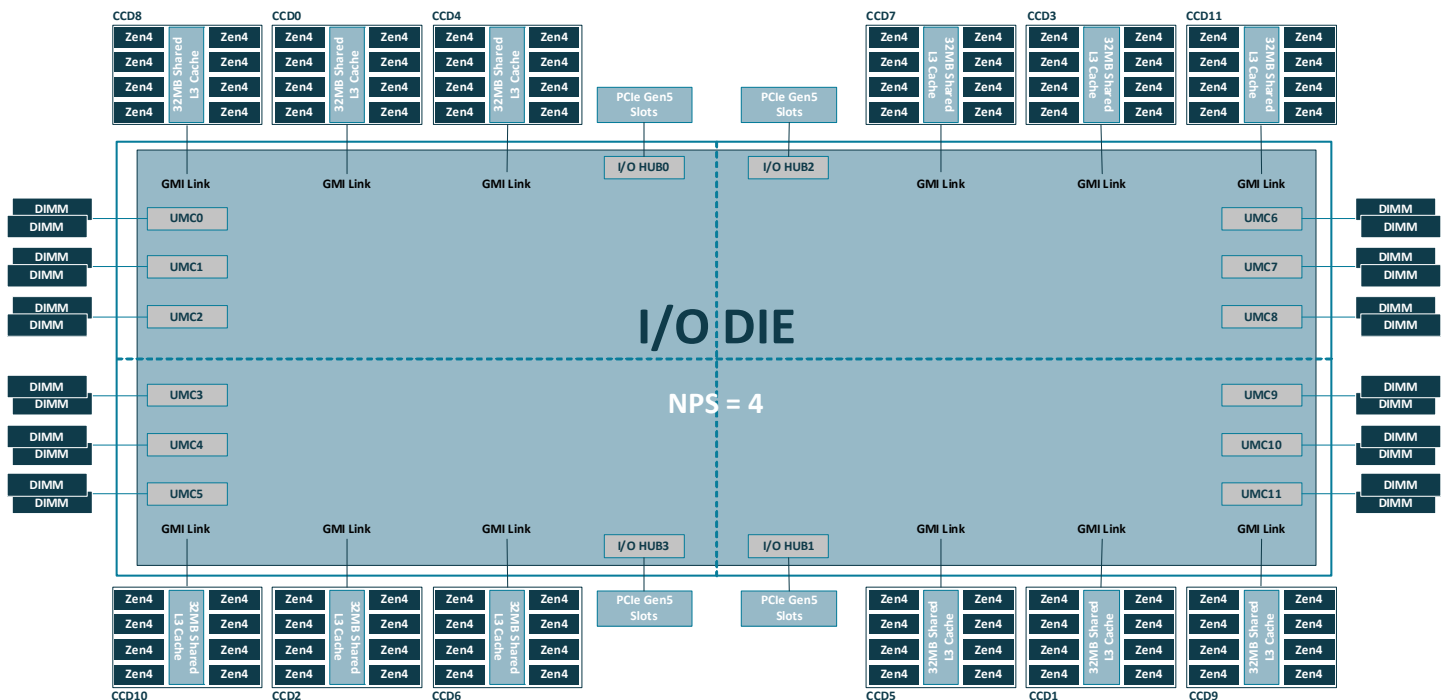


Figure 2-7: The AMD EPYC 9004 SoC consists of up to 12 CCDs and a central IOD for 91xx-96xx models, including “X” OPNs

2.11.2 Models 97xx (“Bergamo”)

97xx 4th Gen AMD EPYC 9004 Series Processors with model numbers 97xx have up to 8 CCDs that each contain two CCXs, as shown below.

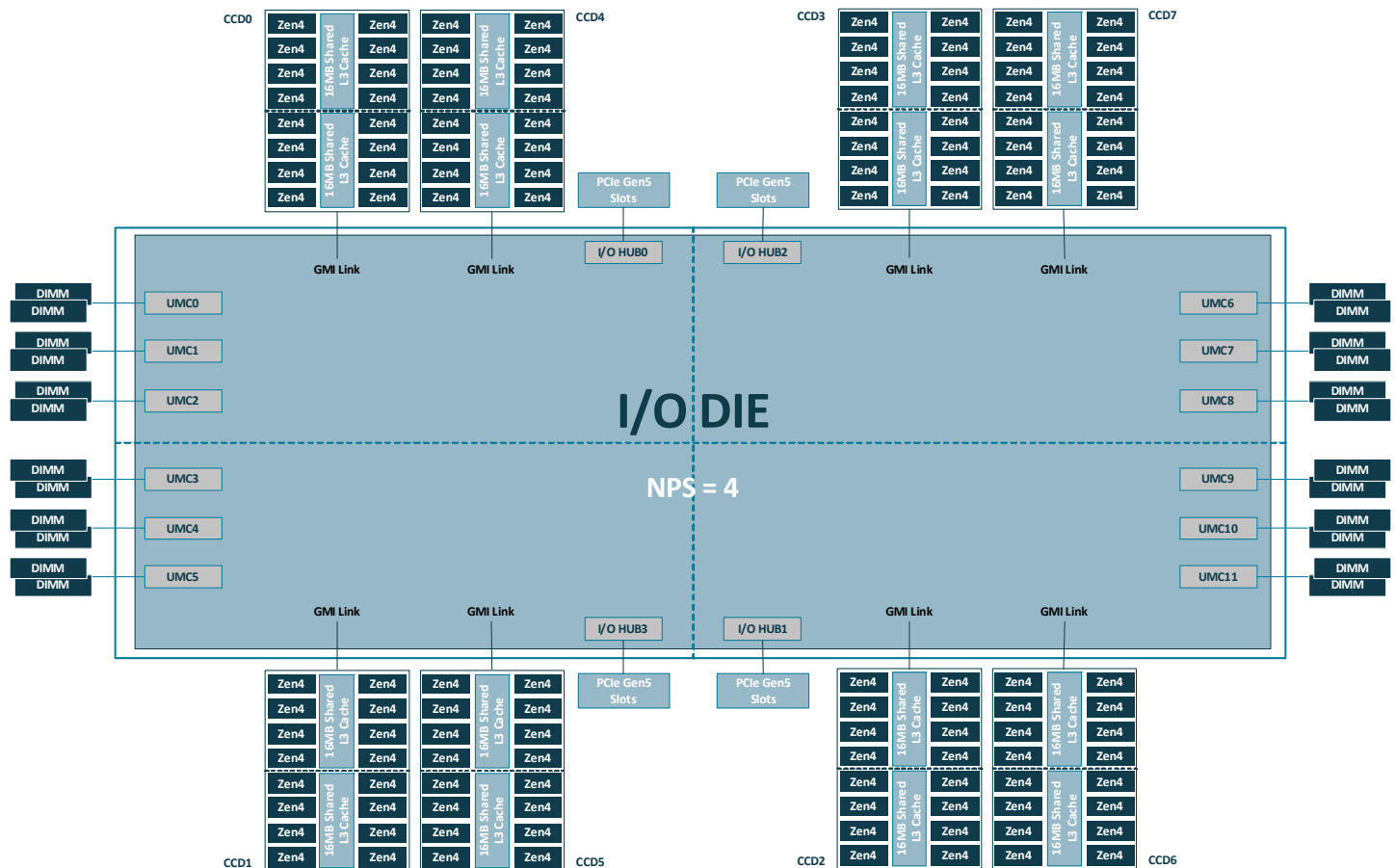


Figure 2-8: The AMD EPYC 9004 System on Chip (SoC) consists of up to 8 CCDs and a central IOD for 97xx models

2.12 NUMA Topology

AMD EPYC 9004 Series Processors use a Non-Uniform Memory Access (NUMA) architecture where different latencies may exist depending on the proximity of a processor core to memory and I/O controllers. Using resources within the same NUMA node provides uniform good performance, while using resources in differing nodes increases latencies.

2.12.1 NUMA Settings

A user can adjust the system **NUMA Nodes Per Socket (NPS)** BIOS setting to optimize this NUMA topology for their specific operating environment and workload. For example, setting NPS=4 as shown in [“Memory and I/O” on page 8](#) divides the processor into quadrants, where each quadrant has 3 CCDs, 3 UMCs, and 1 I/O Hub. The closest processor-memory I/O distance is between the cores, memory, and I/O peripherals within the same quadrant. The furthest distance is between a core and memory controller or IO hub in cross- diagonal quadrants (or the other processor in a 2P configuration). The locality of cores, memory, and IO hub/devices in a NUMA-based system is an important factor when tuning for performance.

The NPS setting also controls the interleave pattern of the memory channels within the NUMA Node. Each memory channel within a given NUMA node is interleaved. The number of channels interleaved decreases as the NPS setting gets more granular. For example:

- A setting of NPS=4 partitions the processor into four NUMA nodes per socket with each logical quadrant configured as its own NUMA domain. Memory is interleaved across the memory channels associated with each quadrant. PCIe devices will be local to one of the four processor NUMA domains, depending on the IOD quadrant that has the corresponding PCIe root complex for that device.
- A setting of NPS=2 configures each processor into two NUMA domains that groups half of the cores and half of the memory channels into one NUMA domain, and the remaining cores and memory channels into a second NUMA domain. Memory is interleaved across the six memory channels in each NUMA domain. PCIe devices will be local to one of the two NUMA nodes depending on the half that has the PCIe root complex for that device.
- A setting of NPS=1 indicates a single NUMA node per socket. This setting configures all memory channels on the processor into a single NUMA node. All processor cores, all attached memory, and all PCIe devices connected to the SoC are in that one NUMA node. Memory is interleaved across all memory channels on the processor into a single address space.
- A setting of NPS=0 indicates a single NUMA domain of the entire system (across both sockets in a two-socket configuration). This setting configures all memory channels on the system into a single NUMA node. Memory is interleaved across all memory channels on the system into a single address space. All processor cores across all sockets, all attached memory, and all PCIe devices connected to either processor are in that single NUMA domain.

You may also be able to further improve the performance of certain environments by using the **LLC (L3 Cache) as NUMA** BIOS setting to associate workloads to compute cores that all share a single LLC. Enabling this setting equates each shared L3 or CCX to a separate NUMA node, as a unique L3 cache per CCD. A single AMD EPYC 9004 Series Processor with 12 CCDs can have up to 12 NUMA nodes when this setting is enabled.

Thus, a single EPYC 9004 Series Processor may support a variety of NUMA configurations ranging from one to twelve NUMA nodes per socket.

Note: If software needs to understand NUMA topology or core enumeration, it is imperative to use documented Operating System (OS) APIs, well-defined interfaces, and commands. Do not rely on past assumptions about settings such as APICID or CCX ordering.

2.13 Dual-Socket Configurations

AMD EPYC 9004 Series Processors support single- or dual-socket system configurations. Processors with a 'P' suffix in their name are optimized for single-socket configurations (see the “Processor Identification” chapter) only. Dual-socket configurations require both processors to be identical. You cannot use two different processor Ordering Part Numbers (OPNs) in a single dual-socket system.

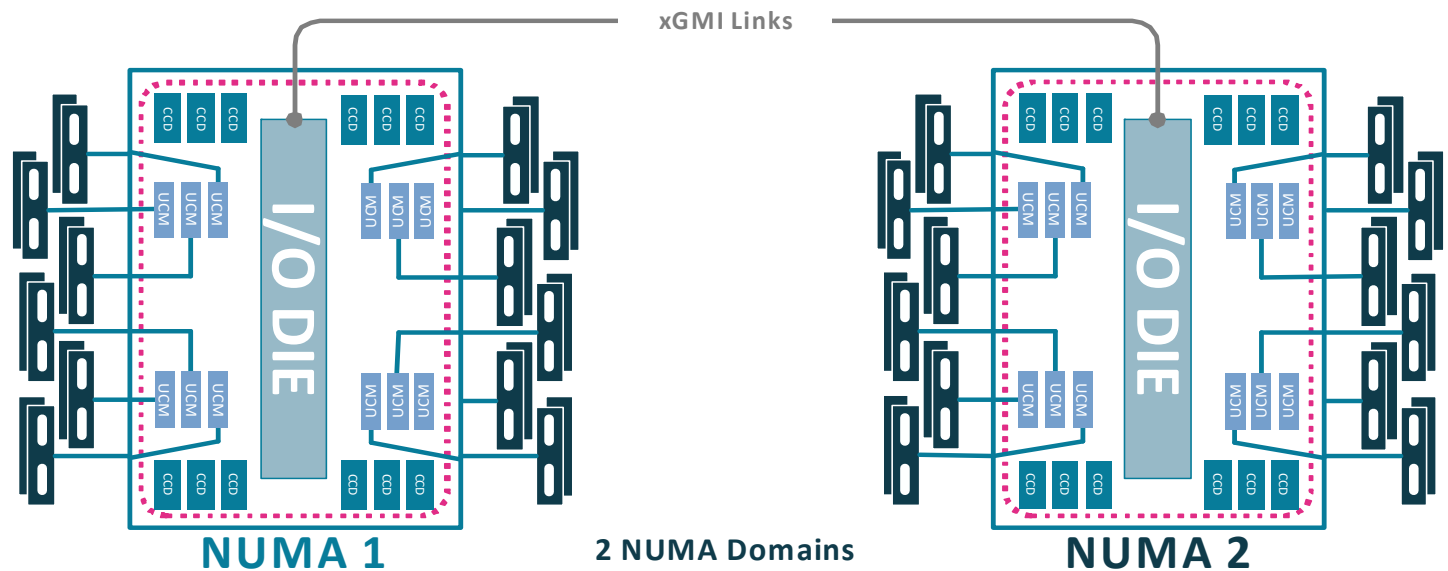


Figure 2-9: Two EPYC 9004 Processors connect through 4 xGMI links (NPS1)

In dual-socket systems, two identical EPYC 9004 series SoCs are connected via their corresponding External Global Memory Interconnect [xGMI] links. This creates a high bandwidth, low latency interconnect between the two processors. System manufacturers can elect to use either 3 or 4 of these Infinity Fabric links depending upon I/O and bandwidth system design objectives.

The Infinity Fabric links utilize the same physical connections as the PCIe lanes on the system. Each link uses up to 16 PCIe lanes. A typical dual socket system will reconfigure 64 PCIe lanes (4 links) from each socket for Infinity Fabric connections. This leaves each socket with 64 remaining PCIe lanes, meaning that the system has a total of 128 PCIe lanes. In some cases, a system designer may want to expose more PCIe lanes for the system by reducing the number of Infinity Fabric G-Links from 4 to 3. In these cases, the designer may allocate up to 160 lanes for PCIe (80 per socket) by utilizing only 48 lanes per socket for Infinity Fabric links instead of 64.

A dual-socket system has a total of 24 memory channels, or 12 per socket. Different OPNs can be configured to support a variety of NUMA domains.

Chapter

3

BIOS Defaults Summary

This chapter provides high-level lists of the default AMD EPYC 9004 BIOS settings and their default values. Please see Chapter 4 of the *BIOS & Workload Tuning Guide for AMD EPYC™ 9004 Series Processors* (available from [AMD EPYC Tuning Guides](#)) for detailed descriptions. Later chapters in this Tuning Guide discuss the BIOS options as they relate to a specific workload or set of workloads.

Note: The default setting names and values described in this chapter are the AMD default names and values that serve as recommendations for OEMs. End users must confirm their OEM BIOS setting availability and options.

AMD strongly recommends that customers download and install the latest BIOS update for your AMD EPYC 9004 Series Processor-based server from your platform vendor. BIOS updates often help customers by providing new and updated features, bug fixes, enhancements, security features, and other improvements. These improvements can help your system software stability and dependency modules (such as hardware, firmware, drivers, and software) by giving you a more robust environment to run your applications.

3.1 Processor Core Settings

Name	Default	Description
SMT Control	Auto	Enabled/Auto: Two hardware threads per core. Disabled: Single hardware thread per core.
L1 Stream HW Prefetcher	Auto	Enabled/Auto: Enables the prefetcher. Disabled: Disables the prefetcher.
L1 Stride Prefetcher	Auto	Enabled/Auto: Enables the prefetcher. Disabled: Disables the prefetcher.
L1 Region Prefetcher	Auto	Enabled/Auto: Enables the prefetcher. Disabled: Disables the prefetcher.
L1 Burst Prefetch Mode	Auto	Enabled/Auto: Enables the prefetcher. Disabled: Disables the prefetcher.
L2 Stream HW Prefetcher	Auto	Enabled/Auto: Enables the prefetcher. Disabled: Disables the prefetcher.
L2 Up/Down Prefetcher	Auto	Enabled/Auto: Enables the prefetcher. Disabled: Disables the prefetcher.
Core Performance Boost	Auto	Enabled/Auto: Enables Core Performance Boost. Disabled: Disables Core Performance Boost.
BoostFmaxEn	Auto	Auto: Use the default Fmax Manual: User can set the boost Fmax
BoostFmax	Auto	Specify the boost Fmax frequency limit to apply to all cores (MHz in decimal)
Global C-State Control	Auto	Enabled/Auto: Controls IO based C-state generation and DF C-states, including core processor C-States Disabled: AMD strongly recommends not disabling this option because this also disables core processor C-States.

Table 3-1: Processor core BIOS settings

X3D	Auto	Enables or disables AMD 3D V-Cache™ technology on Cache Optimized (9004X) processors. • Auto: Enabled on an AMD EPYC 9004 Series processor with AMD 3D V-Cache™ technology, enabling this option enables the AMD 3D V-Cache module in the CCD to increase the total size of the L3 cache memory to 96MB • Disabled: Disabling this option reduces the L3 cache in the CCD to 32MB. <i>Note: This option only applies to AMD EPYC 9004 Series Processors with AMD 3D V-Cache technology.</i> <i>Note: AMD engineers performed extensive internal testing and validation for various applications using the X3D BIOS option found in AMD EPYC 9xx4X processors with AMD 3D V-Cache technology. This testing and validation cannot cover all applications or use cases. Testing has shown AMD 3D V-Cache to be beneficial for most workloads, however AMD recommends that you test and evaluate the benefits of enabling or disabling the X3D BIOS option for your application workloads in your environment and proceeding based on those results.</i>
-----	------	---

Table 3-1: Processor core BIOS settings

3.2 Power Efficiency Settings

Name	Default	Description
Power Profile Selection	Auto	Auto/0: High-performance mode 1: Efficiency mode 2: Maximum I/O performance mode
Determinism Control	Auto	Auto: Use default performance determinism settings. Manual: Specify custom performance determinism settings.
Determinism Enable	Auto	Auto: Performance. 1: Power.
TDP Control	Auto	Auto: Use platform- and OPN-default TDP. Manual: Set custom configurable TDP.
TDP	OPN Max	This option appears once the user sets the TDP Control to Manual . Values 85-400: Set configurable TDP, in watts.
PPT Control	Auto	Enables or disables the PPT control. Auto: Automatically set PPL in watts. Manual: Specify a custom PPL.
PPT	OPN Max	This option appears once the user sets the PPT Control to Manual . Values 85-400: Set configurable PPT, in watts.
CPPC	Auto	Enabled/Auto: Allows the OS to make performance/power optimization requests using ACPI CPPC. Disabled: Prevents the OS from making performance/power optimization requests using ACPI CPPC.

Table 3-2: Power efficiency BIOS settings

3.3 NUMA and Memory Settings

Name	Default	Description
LLC as NUMA Domain (ACPI SRAT L3 Cache as NUMA Domain)	Disabled	Disabled (recommended): Both NUMA nodes (<code>cpubind</code>) and memory interleaving (<code>membind</code>) are determined by the NPS setting. Enabled: Overrides the NPS setting for # of NUMA nodes by mapping each LLC as a NUMA node. This does not impact the memory interleaving
Nodes Per Socket (NPS)	1	Memory Interleaving: The NPS setting always determines the memory interleaving regardless of whether LLC as NUMA is Enabled or Disabled. # of NUMA nodes (if LLC as NUMA Domain is Disabled): NPS1/Auto: One NUMA node per socket (Most cloud providers use this as it provides consistent average memory latency to all the accesses within a socket). NPS2: Two NUMA nodes per socket. NPS4: Four NUMA nodes per socket NPS0 (not recommended): Only applicable for dual-socket systems. A single NUMA node is created for the whole two-socket platform. AMD recommends either NPS1 or NPS4 depending on your use case. Windows systems: Make sure that the number of logical processors per NUMA node is ≤ 64. You can do this by using NPS2 or NPS4 instead of the default NPS1.
Memory Target Speed	Auto	Auto: Determine the maximum memory speed based on SPD information from populated DIMMs and platform memory speed support. Alternatively, you can select: Values 3200–5600 MT/s: Run the DRAM memory target clock speed at the specified speed. The DRAM memory target is the DDR rate. Your OEM system default value may vary.
Memory Interleaving	Auto	Auto/Enable: Enables memory interleaving. Disable: Allows for disabling memory interleaving. The NUMA Nodes per Socket setting will be honored regardless of this setting. AMD strongly recommends not disabling this setting because most production deployments benefit from memory interleaving.

Table 3-3: NUMA and memory BIOS settings

3.4 Infinity Fabric Settings

Name	Default	Description
3-4 xGMI Link Max Speed	Auto	12 Gbps 16 Gbps 17 Gbps 18 Gbps 20 Gbps 22 Gbps 23 Gbps 24 Gbps 25 Gbps/Auto 26 Gbps 27 Gbps 28 Gbps 30 Gbps 32 Gbps Your OEM system default value may vary.
xGMI Link Width Control	Auto	Auto: Use the default xGMI link width controller settings. Manual: Specify a custom xGMI link width controller setting.
xGMI Force Link Width Control	Auto	Unforce: Do not force the xGMI to a fixed width. Force: Use the xGMI link to the user-specified width.
xGMI Force Link Width	Auto	0: Force xGMI link width to x4. 1: Force xGMI link width to x8. 2: Force xGMI link width to x16.
xGMI Max Link Width Control	Auto	Auto: Use the default xGMI link width controller settings. Manual: Specify a custom xGMI link with controller setting.
xGMI Max Link Width	Auto	0: Set max xGMI link width to x8. 1: Set max xGMI link width to x16.
APBDIS	Auto	0/Auto: Dynamically switch the Infinity Fabric P-state based on link usage. 1: Enabled fixed Infinity Fabric P-state control.
DfPstate Range Support	Auto	Auto: If this feature is enabled, the range value setting should follow the rule that $\text{MaxDfPstate} \leq \text{MinDfPstate}$. Otherwise, it will not work. Enable: Add the values MaxDfPstate & MinDfPstate. Disable: No MaxDfPstate & MinDfPstate option.

Table 3-4: Infinity Fabric BIOS settings

DF C-States	Auto	Controls DF C-states. • Disabled: Prevents the AMD Infinity Fabric from entering a low-power state. • Enabled/Auto: Allows the AMD Infinity Fabric to enter a low-power state.
-------------	------	--

Table 3-4: Infinity Fabric BIOS settings

3.5 PCIe, I/O, Security, and Virtualization Settings

Name	Default	Description
Local APIC Mode	Auto(0x02)	• xAPIC: Use xAPIC, supports up to 255 cores. • x2APIC: Supports more than 255 cores. • Auto: The system will choose the mode that best fits the number of active cores in the system. • Compatibility: Threads below 255 run in xAPIC with xAPIC ACPI structures, and threads 255 and above run in x2 mode with x2 ACPI structures. • XApicMode (0x01): Forces legacy xAPIC mode. • X2ApicMode (0x02): Forces x2APIC mode independent of thread count.
PCIe Speed PMM Control	Auto	• 0: Dynamic link speed determined by power management functionality. • 1: Static Target Link Speed (Gen4); sets the maximum idle link speed to 16 GT/s. • Auto/2: Static Target Link Speed (Gen5); sets the maximum idle link speed to 32 GT/s, thereby disabling the feature).
PCIe ARI Support (SRIOV)	Auto	• Enabled/Auto: Enables Alternative Routing ID interpretation. • Disabled: Disables Alternative Routing ID interpretation.
PCIe Ten Bit Tag Support	Auto	• Enabled/Auto: Enables PCIe 10-bit tags for supported devices. • Disabled: Disables PCIe 10-bit tags for all devices.
IOMMU	Auto	• Enabled/Auto: Enables IOMMU. AMD recommends setting this to <code>pt:pass-through</code> in the Linux kernel settings. • Disabled: Disables IOMMU.
AVIC	Disabled	Advanced Virtual Interrupt Controller. • Disabled: Disables AVIC. • Enabled: Enables AVIC.
x2AVIC	Disabled	x2AVIC is an extension of the advanced virtual interrupt controller. This feature currently requires a custom AMD Linux kernel. • Disabled: Disables x2AVIC. • Enabled: Enables x2AVIC.

Table 3-5: PCIe, I/O, security, and virtualization BIOS settings

TSME	Auto	• Auto/Disabled: Disables transparent secure memory encryption. • Enabled: Enables transparent secure memory encryption.
SEV	Disabled	In a multi-tenant environment (such as a cloud), Secure Encrypted Virtualization (SEV) mode isolates virtual machines from each other and from the hypervisor. • Disabled: SEV is disabled. • Enabled: SEV is enabled.
SEV-ES	Disabled	Secure Encrypted Virtualization-Encrypted State (SEV-ES) mode extends SEV protection to the contents of the CPU registers by encrypting them when a virtual machine stops running. Combining SEV and SEV-ES can reduce the attack surface of a VM by helping protect the confidentiality of data in memory. • Disabled: SEV-ES is disabled. • Enabled: SEV-ES is enabled.
SEV-SNP	Disabled	Secure Encrypted Virtualization-Secure Nested Paging (SEV-SNP) mode builds on SEV and SEV-ES by adding strong memory integrity protection to create an isolated execution environment that helps prevent malicious hypervisor-based attacks such as data replay and memory re-mapping. SEV-SNP also introduces several additional optional security enhancements that support additional VM use models, offer stronger protection around interrupt behavior, and increase protection against recently-disclosed side channel attacks. • Disabled: SEV-SNP is disabled. • Enabled: SEV-SNP is enabled.

Table 3-5: PCIe, I/O, security, and virtualization BIOS settings

3.6 Higher-Level Settings

The system powers on to an initial state, after which succeeding software layers may affect system settings:

1. System firmware validates basic hardware functionality and allows users to change various settings via the BIOS Setup menus.
2. UEFI provides a shell environment that allows users to further interact with the system.
3. The operating system or hypervisor is the next software layer that provides control over system hardware.
4. Lastly, certain applications can also affect underlying hardware.

Each of the lines above may alter settings made by prior line, and some user changes require a reboot to take effect.

Please refer to your OEM documentation and/or applicable AMD Tuning Guide(s) for further guidance.

Chapter

4

Common Linux Kernel Tools and Examples

The Ubuntu kernel always had a NUMA-aware memory and CPU scheduling policy. This support is undergoing continuous improvement and enhancement across all releases. Read the [Ubuntu manpages](#)* of tools controlling NUMA binding using `numactl`, automatic NUMA binding using `numad`, kernel automatic NUMA balancing, and more.

NUMA (Non Uniform Memory Access) systems consist of CPU clusters or CPU groups. Each CPU group is called a NUMAnode, and each NUMA node has its own CPUs, memory, and I/O devices. NUMA nodes connect to memory and I/O devices on remote CPUs via one or more buses (or interconnects). The term NUMA comes from the fact that it is faster to access local memory than memory associated with other NUMA nodes.

NUMA architecture introduces memory access latencies depending on the distance between the CPU and the memory location. System BIOS populates the System Locality Information Table (SLIT) and supplies it to the Linux kernel via the Advanced Configuration and Power Interface (ACPI) and provides the normalized distances between the different NUMA nodes. See [Socket SP5 Platform NUMA Topology for AMD Family 19h Models 10h-1Fh](#) for additional information.

Ubuntu includes the optional user-level `numad` daemon that provides process management and placement advice for efficient CPU and memory usage on systems with NUMA topology.

The NUMA tools in Ubuntu can improve application performance on modern hardware systems. Additional tuning can improve performance even more.

Automatic NUMA balancing provides satisfactory performance in most cases, and the default performance is near optimal. This chapter lists some commonly available Linux management tools and provides some examples. See:

- [“lscpu” on page 22](#)
- [“numactl” on page 23](#)
- [“numastat” on page 24](#)
- [“lstopo / hwloc-ls” on page 24](#)
- [“cpupower” on page 26](#)
- [“CPU Governors” on page 27](#)

4.1 lscpu

`lscpu` gives a quick view of CPU topology with the following information:

- Number of sockets, nodes, cores, and threads present in the system.
- Caches and their sizes.
- NUMA nodes and CPU associations.

For example:

```
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          52 bits physical, 57 bits virtual
Byte Order:             Little Endian
CPU(s):                 384
On-line CPU(s) list:   0-383
Vendor ID:              AuthenticAMD
Model name:             AMD EPYC 9654 96-Core Processor
CPU family:             25
Model:                  17
Thread(s) per core:    2
Core(s) per socket:    96
Socket(s):              2
Stepping:               1
Frequency boost:        enabled
CPU max MHz:            2400.0000
CPU min MHz:            1500.0000
BogoMIPS:               4800.15
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca
cmov pat pse36 clflush mmx fxsr sse sse2 ht syscall nx mmxext fxsr_opt pdpe1gb rdtscp lm
constant_tsc rep_good nopl nonstop_tsc cpuid extd_apicid aperfmperf rapl pni pclmulqdq
monitor ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt aes xsave avx f16c rdrand
lahf_lm cmp_legacy svm extapic cr8_legacy abm sse4a misalignsse 3dnowprefetch osvw ibs
skinit wdt tce topoext perfctr_core perfctr_nb bpext perfctr_llc mwaitx cpb cat_l3 cdp_l3
invpcid_single hw_pstate ssbd mba ibrs ibpb stibp vmmcall fsgsbase bmi1 avx2 smep bmi2 erms
invpcid cqm rdt_a avx512f avx512dq rdseed adx smap avx512ifma clflushopt clwb avx512cd
sha_ni avx512bw avx512vl xsaveopt xsavec xgetbv1 xsaves cqm_llc cqm_occup_llc
cqm_mbm_total cqm_mbm_local avx512_bf16 clzero irperf xsaveerptr rdpru wbnoinvd amd_ppin
arat npt lbrv svm_lock nrip_save tsc_scale vmcb_clean flushbyasid decodeassists
pausefilter pfthreshold avic v_vmsave vmload vgif v_spec_ctrl avx512vbmi umip pku ospke
avx512_vbmi2 gfni vaes vpclmulqdq avx512_vnni avx512_bitalg avx512_vpopcntdq la57 rdpid
overflow_recov succor smca fsrm flush_lld
Virtualization:         AMD-V
L1d cache:              6 MiB (192 instances)
L1i cache:              6 MiB (192 instances)
L2 cache:               192 MiB (192 instances)
L3 cache:               768 MiB (24 instances)
NUMA node(s):           24
NUMA node0 CPU(s):      0-7,192-199
NUMA node1 CPU(s):      24-31,216-223
NUMA node2 CPU(s):      48-55,240-247
NUMA node3 CPU(s):      72-79,264-271
NUMA node4 CPU(s):      8-15,200-207
NUMA node5 CPU(s):      32-39,224-231
NUMA node6 CPU(s):      56-63,248-255
NUMA node7 CPU(s):      80-87,272-279
NUMA node8 CPU(s):      16-23,208-215
NUMA node9 CPU(s):      40-47,232-239
NUMA node10 CPU(s):     64-71,256-263
NUMA node11 CPU(s):     88-95,280-287
```

```

NUMA node12 CPU(s) :          96-103,288-295
NUMA node13 CPU(s) :          120-127,312-319
NUMA node14 CPU(s) :          144-151,336-343
NUMA node15 CPU(s) :          168-175,360-367
NUMA node16 CPU(s) :          104-111,296-303
NUMA node17 CPU(s) :          128-135,320-327
NUMA node18 CPU(s) :          152-159,344-351
NUMA node19 CPU(s) :          176-183,368-375
NUMA node20 CPU(s) :          112-119,304-311
NUMA node21 CPU(s) :          136-143,328-335
NUMA node22 CPU(s) :          160-167,352-359
NUMA node23 CPU(s) :          184-191,376-383
Vulnerability Itlb multihit:    Not affected
Vulnerability L1tf:            Not affected
Vulnerability Mds:             Not affected
Vulnerability Meltdown:        Not affected
Vulnerability Mmio stale data:  Not affected
Vulnerability Retbleed:        Not affected
Vulnerability Spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl
and seccomp
Vulnerability Spectre v1:       Mitigation; usercopy/swapgs barriers and __user pointer
sanitization
Vulnerability Spectre v2:       Mitigation; Retpolines, IBPB conditional, IBRS_FW, STIBP
always-on, RSB filling, PBRSE-eIBRS Not affected
Vulnerability Srbds:           Not affected
Vulnerability Tsx async abort:  Not affected

```

4.2 numactl

`numactl` can control the NUMA policy for processes or shared memory. `numactl` and `numad` help tune NUMA scheduling parameters and monitor both NUMA topology and resource utilization by automatically making affinity adjustments to locally optimized processes. You can use `numactl` to find:

- Number of NUMA nodes in the system.
- Distance between nodes.
- CPU and memory association with the node.
- Linux policy (default, bind, preferred, interleave) of the current process.

The second part of the `numactl --hardware` output gives the node distances in a matrix based on the System Locality Information Table in the Advanced Configuration and Power Interface (ACPI SLIT). These distances indicate the cost of accessing remote memory as the relative latency to access memory from a particular node, normalized to a base value of 10. Higher values indicate more overhead.

For example:

```

# If not already present install with
4.2.1$ sudo apt install numactl

# Show the hardware numa characteristics
$ sudo numactl --hardware
available: 2 nodes (0-1)
node 0 cpus: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58
59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88
89 90 91 92 93 94 95 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209
210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232
233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255

```

```

256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278
279 280 281 282 283 284 285 286 287
node 0 size: 31718 MB
node 0 free: 25038 MB
node 1 cpus: 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116
117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139
140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185
186 187 188 189 190 191 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304
305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327
328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350
351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373
374 375 376 377 378 379 380 381 382 383
node 1 size: 0 MB
node 1 free: 0 MB
node distances:
node    0    1
  0:   10   32
  1:   32   10

```

4.3 numastat

You can execute the `numastat` command to view the general system NUMA statistics, including the most important factor of hits/misses to local memory.

```

$ numastat

```

	node0	node1
numa_hit	16503206	0
numa_miss	0	0
numa_foreign	0	0
interleave_hit	443936	0
local_node	8544711	0
other_node	7958495	0

This example shows a rather idle system that is not yet using the second node and is therefore not yet experiencing any problems with having local process memory being allocated on another node (`numa_miss/numa_foreign`).

4.4 lstopo / hwloc-ls

`hwloc` provides both command line tools and a C API to obtain the hierarchical map of key computing elements, such as NUMA memory nodes, shared caches, processor packages, processor cores, and processor threads.

`hwloc` includes various options that use the syntax `hwloc-<option>`. The following example shows a shortened overview of an AMD 9004 Series Processor. This `lstopo` example does not begin with a `hwloc-` prefix and is the same as executing the command `hwloc-ls`.

```

# Install if not already available
$ sudo apt install hwloc-nox

$ lstopo
Machine (31GB total)
  Package L#0
    NUMANode L#0 (P#0 31GB)
    L3 L#0 (32MB)
      L2 L#0 (1024KB) + L1d L#0 (32KB) + L1i L#0 (32KB) + Core L#0
        PU L#0 (P#0)
        PU L#1 (P#192)
      L2 L#1 (1024KB) + L1d L#1 (32KB) + L1i L#1 (32KB) + Core L#1

```

```

    PU L#2 (P#1)
    PU L#3 (P#193)
    L2 L#2 (1024KB) + L1d L#2 (32KB) + L1i L#2 (32KB) + Core L#2
    PU L#4 (P#2)
    PU L#5 (P#194)
...
    L2 L#92 (1024KB) + L1d L#92 (32KB) + L1i L#92 (32KB) + Core L#92
    PU L#184 (P#92)
    PU L#185 (P#284)
    L2 L#93 (1024KB) + L1d L#93 (32KB) + L1i L#93 (32KB) + Core L#93
    PU L#186 (P#93)
    PU L#187 (P#285)
    L2 L#94 (1024KB) + L1d L#94 (32KB) + L1i L#94 (32KB) + Core L#94
    PU L#188 (P#94)
    PU L#189 (P#286)
    L2 L#95 (1024KB) + L1d L#95 (32KB) + L1i L#95 (32KB) + Core L#95
    PU L#190 (P#95)
    PU L#191 (P#287)
HostBridge
PCIBridge
    PCI 01:00.0 (NVMEExp)
        Block(Disk) "nvme0n1"
HostBridge
PCIBridge
    PCIBridge
        PCI 62:00.0 (VGA)
PCIBridge
    PCI 63:00.0 (Ethernet)
        Net "enp99s0"
Package L#1
    L3 L#12 (32MB)
        L2 L#96 (1024KB) + L1d L#96 (32KB) + L1i L#96 (32KB) + Core L#96
        PU L#192 (P#96)
        PU L#193 (P#288)
...
    L2 L#190 (1024KB) + L1d L#190 (32KB) + L1i L#190 (32KB) + Core L#190
    PU L#380 (P#190)
    PU L#381 (P#382)
    L2 L#191 (1024KB) + L1d L#191 (32KB) + L1i L#191 (32KB) + Core L#191
    PU L#382 (P#191)
    PU L#383 (P#383)
HostBridge
PCIBridge
    2 x { PCI e2:00.0-1 (SATA) }

```

This command can also generate graphical output if you install the `sudo apt install hwloc` option that consists of nested boxes representing the inclusion of objects in the resource hierarchies. In this graphical output, a **Machine** box usually contains one or several **Package** boxes that contain multiple **Core** boxes that each have one or more PUs.

Caches appear in a slightly different manner because they do not actually include computing resources, such as cores. For instance, a L2 Cache shared by a pair of cores appears as a **Cache** box on top of two **Core** boxes (instead of having **Core** boxes inside the **Cache** box).

By default, NUMA node boxes appear on top of their local computing resources. For instance, a processor **Package** containing one NUMA node and four **Cores** appears as a **Package** box containing the NUMA node box above four **Core** boxes. If a NUMA node is local to the L3 Cache, then the NUMA node appears above that **Cache** box.

the middle. Please see the following figure and <https://github.com/open-mpi/hwloc/tree/master/utils/lstopo>* for additional information.

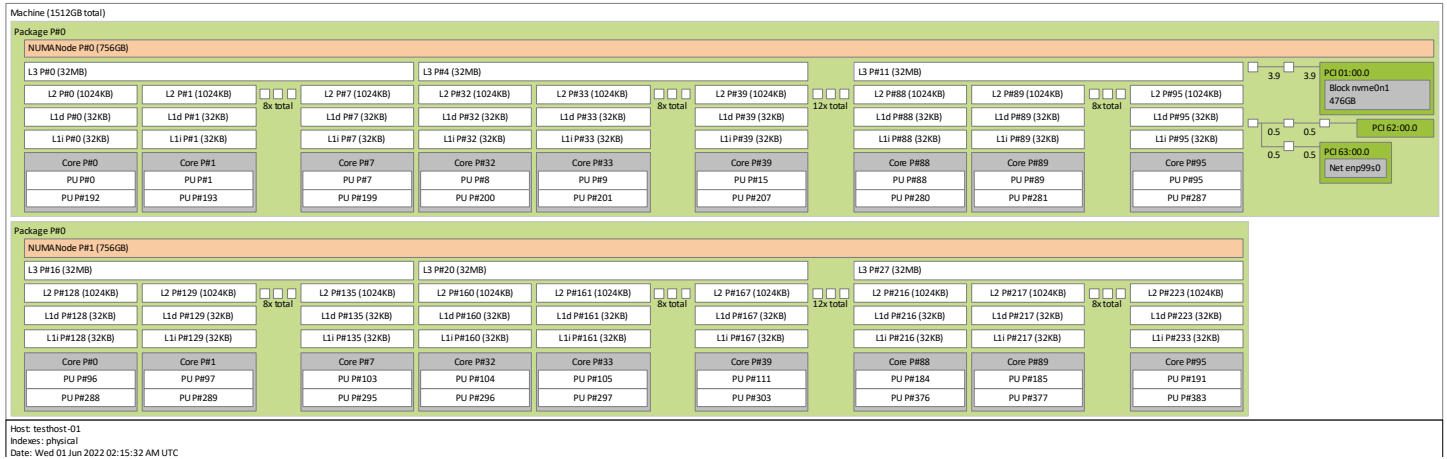


Figure 4-1: Sample graphical Istopo output

4.4.1 hwloc-info

hwloc-info gets the system cache hierarchy.

```
$ sudo hwloc-info
depth 0:      1 Machine (type #0)
depth 1:      24 Package (type #1)
depth 2:      24 L3Cache (type #6)
depth 3:      192 L2Cache (type #5)
depth 4:      192 L1dCache (type #4)
depth 5:      192 L1iCache (type #9)
depth 6:      192 Core (type #2)
depth 7:      384 PU (type #3)
Special depth -3: 1 NUMANode (type #13)
Special depth -4: 8 Bridge (type #14)
Special depth -5: 5 PCIDev (type #15)
Special depth -6: 2 OSDev (type #16)
```

4.5 cpupower

`cpupower` is a collection of tools that examine and tune processor power-saving features, such as checking the CPU governor. The `cpupower monitor` command samples the current CPU frequency in real time. From the Linux perspective, the CPU frequency scaling subsystem can modulate the CPU frequency based on workload feedback. Setting the governor to `performance` reduces OS frequency toggling. See [CPU Performance Scaling](#)* for additional information.

```
# Install if not yet available
$ sudo apt install linux-tools-common

$ sudo cpupower frequency-info
analyzing CPU 0:
  driver: amd-pstate
  CPUs which run at the same hardware frequency: 0
  CPUs which need to have their frequency coordinated by software: 0
```

```

maximum transition latency: 131 us
hardware limits: 400 MHz - 3.52 GHz
available cpufreq governors: conservative ondemand userspace powersave performance
schedutil
current policy: frequency should be within 400 MHz and 3.52 GHz.
                  The governor "schedutil" may decide which speed to use
                  within this range.
current CPU frequency: Unable to call hardware
current CPU frequency: 1.19 GHz (asserted by call to kernel)
boost state support:
  Supported: yes
  Active: yes
  Boost States: 0
  Total States: 3
  Pstate-P0: 1200MHz
  Pstate-P1: 1200MHz
  Pstate-P2: 1200MHz

```

Execute `cat /sys/devices/system/cpu/cpufreq/boost` to check whether boost is ON (1) or OFF (0).

4.5.1 cpupower monitor

Check core frequencies and idle states.

```

$ sudo cpupower monitor

```

PKG	CORE	CPU	Mperf		Freq	Idle_Stats		
			C0	Cx		POLL	C1	C2
0	0	0	0.26	99.74	1159	0.00	0.00	100.9
0	0	192	0.01	99.99	1173	0.00	0.00	100.2
0	1	1	0.01	99.99	1163	0.00	0.00	100.4
0	1	193	0.02	99.98	1181	0.00	0.00	100.1
0	2	2	0.03	99.97	1158	0.00	0.00	100.4
0	2	194	0.03	99.97	1172	0.00	0.00	100.1
0	3	3	0.01	99.99	1143	0.00	0.00	100.4
0	3	195	0.03	99.97	1245	0.00	0.00	100.1
0	4	4	0.03	99.97	1361	0.00	0.00	100.4
0	4	196	0.02	99.98	1624	0.00	0.00	100.1

4.6 CPU Governors

AMD EPYC supports several CPU governors. Different governors can be applied to different cores. For example, the performance governor is often used in a High-Performance Computing environment.

- **performance:** Sets the core frequency to the highest available frequency within P0.
- **Boost=OFF:** The CPU will operate at the base frequency for the specific CPU model.
- **Boost=ON:** The CPU will attempt to boost the frequency up to the Max Boost frequency of 3.4Ghz. While operating at the boosted frequencies, this still represents the P0 P-state.
- **ondemand:** Sets the core frequency depending on the trailing load. This favors a rapid ramp to the highest operating frequency with a subsequent slow step down to P2 when idle. This could penalize short-lived threads.
- **conservative:** Similar to `ondemand` but favors a more graceful ramp to highest frequency and a rapid return to P2 at idle.
- **powersave:** Sets the lowest-supported core frequency, locking it to P2.

Administrators can execute the `cpupower` command to set the CPU governor. For example, to set the CPU governor to Performance:

```
cpupower frequency-set -g performance
```

Please see [Linux CPUFreq Governors](#)* for a more extensive discussion and explanation of Linux CPU governors.

Here are some more examples* of using `cpupower` to query and set a range of conditions on the CPU:

```
cpupower -c 0-15 monitor
```

Displays the frequencies of cores 0 to 15. Useful if a user needs to observe the changes while turning Boost ON and OFF.

```
cpupower frequency-info
```

Lists the boost state, CPU governor, and other useful information about the CPU configuration.

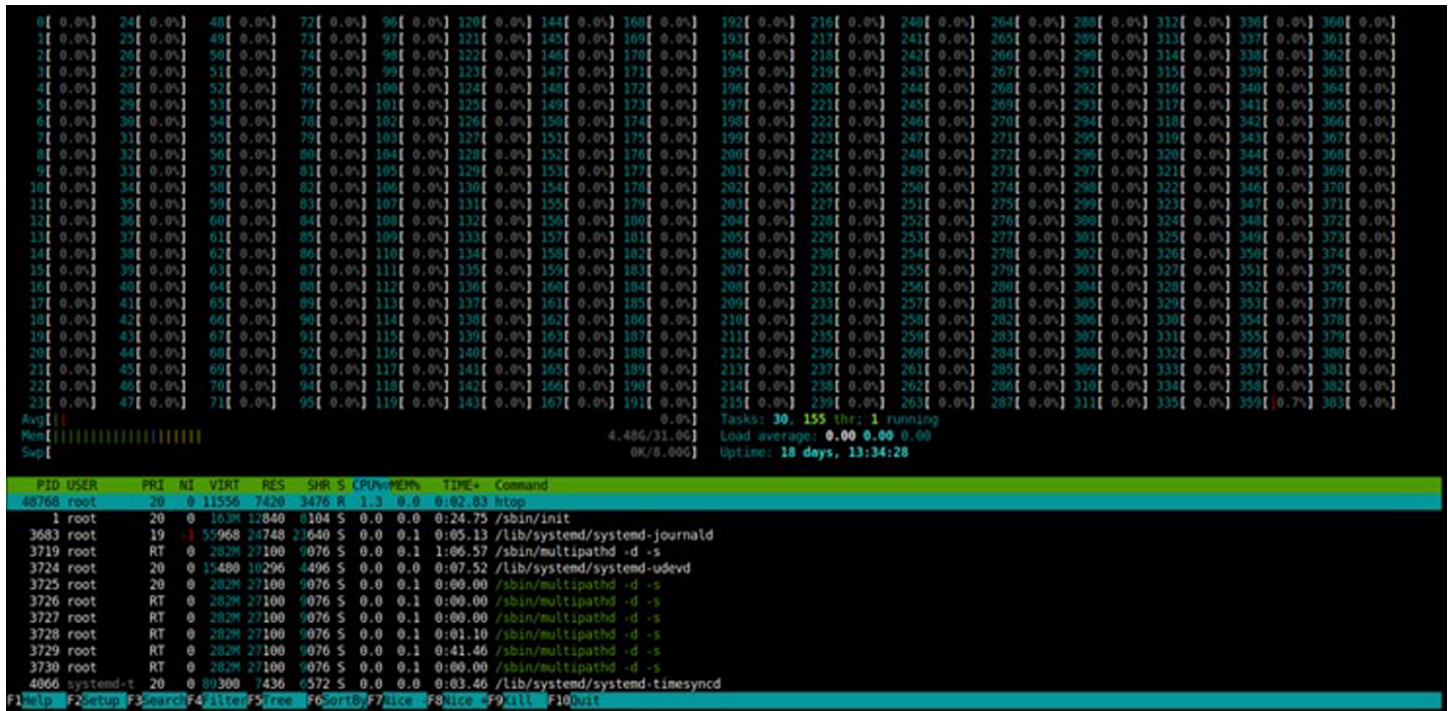
```
sudo cpupower --cpu -c 0-15 idle-set -disable 2
```

Disables the C2 idle state on CPUs 0 to 15.

4.7 htop

`htop` provides a dynamic view of the resources being consumed by various processes. While running `htop`, you can press [F2] and then configure its appearance in various ways. For example, for large systems powered by high-core-count CPUs, you could choose the **CPUs (1-8/16): first half in 8 shorter columns and CPUs (9-16/16): second half in 8 shorter columns** option to more easily see results.

4.7.1 Example 1: Per-CPU Utilization Statistics



4.7.2 Example 2: Per-NUMA-Node Utilization Statistics

The older (and in many other cases less powerful) `top` program still has a NUMA-related feature that `htop` is missing. While running, you can:

- Press [2] to view per-NUMA-node utilization statistics.
- Press [3] to select and highlight a NUMA node and view summary information.

```
top - 10:11:57 up 18 days, 13:39, 2 users, load average: 0.00, 0.00, 0.00
Tasks: 3192 total, 1 running, 3191 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
%Node0 : 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
%Node1 : 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 31718.9 total, 24904.4 free, 4490.3 used, 2324.3 buff/cache
MiB Swap: 8192.0 total, 8192.0 free, 0.0 used. 26711.7 avail Mem
.....
```

This page intentionally left blank.

Chapter

5

General Tuning Recommendations

5.1 LLC as NUMA Domain

Certain datacenter applications that use a remote job scheduler to manage workloads can benefit from pinning execution to a single NUMA node and (preferably) share a single Last-Level Cache (LLC or L3 cache) within that node. This can be done if the system BIOS includes the L3AsNumaNode setting, which creates a NUMA node for each system CCX (L3 cache) when enabled.

Enabling this setting can improve performance for highly NUMA-optimized workloads if either the workloads or components of the workloads can:

- Be pinned to cores in a CCX.
- Benefit from sharing an L3 cache.

Please see [Socket SP5 Platform NUMA Topology for AMD Family 19h Models 10h-1Fh](#) - Login required; please review the latest version if multiple versions are present. This manual contains additional information about NUMA architecture and settings for AMD EPYC 9004 Series Processors.

1P System AMD EPYC 9004 Series Processor with 8 CCDs	# NUMA Nodes with LLCasNUMA is		Memory Interleaving when LLCasNUMA is (Enabled or Disabled)
	Enabled	Disabled	
NPS1	# of CCDs	1	Across all the channels in a socket.
NPS2	# of CCDs	2	All the channels are divided in two groups provided these channels have DIMM populated in them.
NPS4	# of CCDs	4	All the channels are divided in four groups. Across the channels in a group provided these channels have DIMM populated in them.

Table 5-1: # of NUMA nodes and memory interleaving for a single AMD EPYC 9004 Series Processor

2P System AMD EPYC 9004 Series Processor with 8 CCDs	# NUMA Nodes with LLCasNUMA is		Memory Interleaving when LLCasNUMA is (Enabled or Disabled)
	Enabled	Disabled	
NPS0	2 x # of CCDs	1	Across all channels in both sockets. This is not recommended.
NPS1	2 x # of CCDs	2	Across all the channels in each socket.
NPS2	2 x # of CCDs	4	All the channels are divided in two groups in each socket. Across the channels in a group provided these channels have DIMM populated in them.
NPS4	2 x # of CCDs	8	All the channels are divided in four groups in each socket. Across the channels in a group provided these channels have DIMM populated in them.

Table 5-2: # of NUMA nodes and memory interleaving for dual AMD EPYC 9004 Series Processors

Please see [Memory Population Guidelines for AMD EPYC 9004 Series Processors](#) for additional information.

Note: Not all AMD EPYC 9004 Series Processors have 8 CCDs.

5.2 AMD uProf

AMD uProf is a performance analysis tool for applications running on Windows and Linux. Developers can use this tool to better understand and find ways to optimize application runtime performance. AMD uProf offers:

- **Performance Analysis:** The CPU profiling utility identifies application runtime performance bottlenecks.
- **System Analysis:** The Performance Counter Monitor utility monitors system performance metrics.
- **Power Profiling:** System-wide power profiling monitors system thermal and power characteristics.
- **Energy Analysis:** The Power Application Analysis utility identifies energy hotspots in Windows applications.

Please see <https://developer.amd.com/amd-uprof/> for more information about AMD uProf.

5.3 perf

`perf` is a powerful tool that helps monitor various OS subsystems at the server, process, or process subset level to detect and identify performance bottlenecks and possibly tune the OS. Here are two examples of `perf` functionality and usage:

- `perf record` samples the function calls executed by a process or processes and writes the output to `perf.data`.
- `perf report` reads the `perf.data` file and prints a human-readable report of `top` function calls grouped by function calls and ordered by count.

For example:

- `perf record -a -e cycles sleep 30` captures 30 seconds of data for the entire system.

```
$ sudo perf record -a -e cycles sleep 30
[ perf record: Woken up 3 times to write data ]
[ perf record: Captured and wrote 1.804 MB perf.data (22175 samples) ]
```

- `perf record -e cycles <command>` gathers profile information for a given workload.

```
$ sudo perf record -e cycles ls -R > /dev/null
[ perf record: Woken up 2 times to write data ]
[ perf record: Captured and wrote 0.016 MB perf.data (9 samples) ]
```

You can also use trace points or create probe points using either `perf` or `trace-cmd` to gather specific information on the OS. Describing these analyses is beyond the scope of this tuning guide. Profiling can be helpful but is considered a security issue because it allows analyzing and also indirectly slows applications. If you do not have sufficient permissions and settings, then the tool will guide you to the appropriate `proc-fs` entries.

Here are a few invocation examples of `perf` commands.

5.3.1 `perf list cpu`

The `perf list cpu` command displays the symbolic events that you can select in the various `perf` commands using the `-e` option.

```
$ sudo perf list cpu
```

List of pre-defined events (to be used in `-e`):

<code>cpu-cycles</code> OR <code>cycles</code>	[Hardware event]
<code>cpu-clock</code>	[Software event]
<code>cpu-migrations</code> OR <code>migrations</code>	[Software event]
<code>branch-instructions</code> OR <code>cpu/branch-instructions/</code>	[Kernel PMU event]
<code>branch-misses</code> OR <code>cpu/branch-misses/</code>	[Kernel PMU event]
<code>cache-misses</code> OR <code>cpu/cache-misses/</code>	[Kernel PMU event]
<code>cache-references</code> OR <code>cpu/cache-references/</code>	[Kernel PMU event]
<code>cpu-cycles</code> OR <code>cpu/cpu-cycles/</code>	[Kernel PMU event]
<code>instructions</code> OR <code>cpu/instructions/</code>	[Kernel PMU event]
<code>stalled-cycles-backend</code> OR <code>cpu/stalled-cycles-backend/</code>	[Kernel PMU event]
<code>stalled-cycles-frontend</code> OR <code>cpu/stalled-cycles-frontend/</code>	[Kernel PMU event]

memory:

<code>ls_ret_cpuid</code>	[The number of CPUID instructions retired]
<code>cpuhp:cpuhp_enter</code>	[Tracepoint event]
<code>cpuhp:cpuhp_exit</code>	[Tracepoint event]
<code>cpuhp:cpuhp_multi_enter</code>	[Tracepoint event]
<code>kmem:mm_page_pcpu_drain</code>	[Tracepoint event]
<code>kvm:kvm_cpuid</code>	[Tracepoint event]
<code>kvm:kvm_vcpu_wakeup</code>	[Tracepoint event]
<code>kvm:vcpu_match_mmio</code>	[Tracepoint event]
<code>percpu:percpu_alloc_percpu</code>	[Tracepoint event]
<code>percpu:percpu_alloc_percpu_fail</code>	[Tracepoint event]

```

percpu:percpu_create_chunk      [Tracepoint event]
percpu:percpu_destroy_chunk     [Tracepoint event]
percpu:percpu_free_percpu      [Tracepoint event]
power:cpu_frequency             [Tracepoint event]
power:cpu_frequency_limits     [Tracepoint event]
power:cpu_idle                  [Tracepoint event]
syscalls:sys_enter_getcpu       [Tracepoint event]
syscalls:sys_exit_getcpu       [Tracepoint event]
xdp:xdp_cpumap_enqueue         [Tracepoint event]
xdp:xdp_cpumap_kthread         [Tracepoint event]
xen:xen_cpu_load_idt           [Tracepoint event]
xen:xen_cpu_set_ldt            [Tracepoint event]
xen:xen_cpu_write_gdt_entry     [Tracepoint event]
xen:xen_cpu_write_idt_entry     [Tracepoint event]
xen:xen_cpu_write_ldt_entry     [Tracepoint event]

```

5.3.2 perf list cache

The `perf list cache` command displays the pre-defined events that you can select in the various `perf` commands using the `-e` option.

```
$ sudo perf list cache
```

List of pre-defined events (to be used in `-e`):

```

L1-dcache-load-misses          [Hardware cache event]
L1-dcache-loads                [Hardware cache event]
L1-dcache-prefetches           [Hardware cache event]
L1-icache-load-misses          [Hardware cache event]
L1-icache-loads                [Hardware cache event]
branch-load-misses             [Hardware cache event]
branch-loads                   [Hardware cache event]
dTLB-load-misses               [Hardware cache event]
dTLB-loads                     [Hardware cache event]
iTLB-load-misses               [Hardware cache event]
iTLB-loads                     [Hardware cache event]

```

5.3.3 perf stat

`perf-stat` executes a command and gathers performance counter statistics using the following syntax:

```

perf stat [-e <EVENT> | --event=EVENT] [-a] <command>
perf stat [-e <EVENT> | --event=EVENT] [-a] - <command> [<options>]
perf stat [-e <EVENT> | --event=EVENT] [-a] record
[- o file] - <command> [<options>]
perf stat report [-i file]

```

Here are a few `perf` examples:

```
$ sudo perf stat -e L1-dcache-loads,L1-dcache-load-misses,L1-dcache-prefetches ls -R > /dev/null
```

Performance counter stats for 'ls -R':

```

      833155      L1-dcache-loads
      27498      L1-dcache-load-misses      #      3.30% of all L1-dcache accesses
       5997      L1-dcache-prefetches

```

```
0.000863468 seconds time elapsed
```

```
0.000990000 seconds user
```

```
0.000000000 seconds sys
```

```
$ sudo perf stat -e dTLB-loads,dTLB-load-misses ls -R > /dev/null

Performance counter stats for 'ls -R':

      3847      dTLB-loads
       546      dTLB-load-misses           #   14.19% of all dTLB cache accesses

0.000793986 seconds time elapsed

0.000918000 seconds user
0.000000000 seconds sys
```

```
$ sudo perf stat -e branch-loads,branch-load-misses ls -R > /dev/null

Performance counter stats for 'ls -R':

    381249      branch-loads
     14238      branch-load-misses

0.000736055 seconds time elapsed

0.000857000 seconds user
0.000000000 seconds sys
```

The core AMD PMU has a 4-bit-wide per-cycle increment for each performance monitor counter. This works for most counters, but AMD EPYC Family 17h and above processors can have more than 15 events occur in a cycle. These events are called “Large Increment per Cycle” events. One example is the number of SSE/AVX FLOPs retired (event code 0x003). To count these events, two adjacent hardware PMCs have their count signals merged to form a total of 8 bits per cycle. The PERF_CTR count registers also merge so as to count up to 64 bits.

Normally, events such as instructions retired get programmed on a single counter. For example:

```
PERF_CTL0 (MSR 0xc0010200) 0x0000000000053ff0c # event 0x0c, umask 0xff
PERF_CTR0 (MSR 0xc0010201) 0x00008000000000001 # r/w 48-bit count
```

The next counter at MSRs 0xc0010202-3 either remains unused or can be used independently to count something else.

When counting Large Increment per Cycle events, such as FLOPs, we have to reserve the next counter and program the PERF_CTL (config) register with the Merge event (0xFFF). For example:

```
PERF_CTL0 (msr 0xc0010200) 0x0000000000053ff03 # FLOPs event, umask 0xff
PERF_CTR0 (msr 0xc0010201) 0x00008000000000001 # read 64-bit count, wr low 48b
PERF_CTL1 (msr 0xc0010202) 0x00000000f004000ff # Merge event, enable bit
PERF_CTR1 (msr 0xc0010203) 0x00000000000000000 # write higher 16-bits of count
```

The count widens from the normal 48 bits to 64 bits by having the second counter carry the higher 16 bits of the count in the lower 16 bits of its counter register. This version does not support mixed 48-bit and 64-bit counts. Here is an example for an AMD EPYC 9004 processor:

```
$ sudo perf stat -e cpu/instructions/,cpu/event=0x03,umask=0xff/ ls -R > /dev/null
```

```
Performance counter stats for 'ls -R':
```

```
      1927309      cpu/instructions/  
         3      cpu/event=0x03,umask=0xff/
```

```
0.000841387 seconds time elapsed
```

```
0.000968000 seconds user
```

```
0.000000000 seconds sys
```

Chapter

6

Virtualization

Ubuntu 20.04.5 and 22.04 includes virtualization functionality that allows a machine running Ubuntu 20.04.5 and 22.04 to host multiple virtual machines (VMs), also referred to as guests. VMs use the host's physical hardware and computing resources to run a separate, virtualized operating system (guest OS) as a user-space process on the host OS.

6.1 Secure Encrypted Virtualization (SEV)

AMD Secure Encrypted Virtualization (SEV) protects the data in DRAM while in use by a running VM instance. SEV encrypts the memory of each instance with a unique key. Executing the following command tells you whether the deployment is SEV-capable:

```
# lscpu | grep -i sev
```

```
Flags:                fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat
pse36 clflush mmx fxsr sse sse2 ht syscall nx mmxext fxsr_opt pdpe1gb rdtscp lm
constant_tsc rep_good nopl nonstop_tsc cpuid extd_apicid aperfmperf pni pclmulqdq
monitor ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt aes xsave avx f16c
rdrand lahf_lm cmp_legacy svm extapic cr8_legacy abm sse4a misalignsse 3dnowprefetch
osvw ibs skinit wdt tce topoext perfctr_core perfctr_nb bpext perfctr_llc mwaitx cpb
cat_l3 cdp_l3 invpcid_single hw_pstate sme ssbd mba sev ibrs ibpb stibp vmmcall
fsgsbase bmi1 avx2 smep bmi2 erms invpcid cqm rdt_a rdseed adx smap clflushopt clwb
sha_ni xsaveopt xsavec xgetbv1 xsaves cqm_llc cqm_occup_llc cqm_mbm_total
cqm_mbm_local clzero irperf xsaveerptr wbnoinvd amd_ppin arat npt lbrv svm_lock
nrip_save tsc_scale vmcb_clean flushbyasid decodeassists pausefilter pfthreshold
v_vmsave_vmload vgif umip pku ospke vaes vpclmulqdq rdpid overflow_recov succor smca
```

If enabled on a virtual machine (VM), then SEV encrypts VM memory, which prevents the host from accessing data on the VM. This increases VM security if the host is successfully breached. The host hardware version determines how many VMs can use this feature simultaneously on a single host.

```
# dmesg | grep -i sev
```

```
[ 0.938380] AMD Memory Encryption Features active: SEV SEV-ES
```

Enabling SEV requires all DMA operations inside the guest to use shared memory. SEV makes this transparent to the guest by using the SWIOTLB Linux kernel pool, which has a default size of 64MB. A guest panic will occur if the Linux kernel exhausts the SWIOTLB pool. The number of devices used by the guest and the utilization of these devices directly impacts the amount of SWIOTLB required. AMD recommends increasing the SWIOTLB pool that the Linux kernel allocates for SEV guests, with 512MB as the recommended starting size.

6.1.1 SEV Prerequisites

To ensure SEV support,

- The deployment must include a compute node that runs on SEV-capable AMD hardware, such as an AMD EPYC CPU.
- The deployment must include `libvirt` 4.5 or later, which includes SEV support. This translates to Ubuntu 20.04 Focal or later that came with `libvirt` v6.0.0
- The operating system running in an encrypted instance must support SEV. For example:

```
# dmesg | grep -i sev
[ 8.723924] ccp 0000:22:00.1: sev enabled
[ 8.820716] ccp 0000:22:00.1: SEV API:1.13 build:13
```

Please see the [AMD Secure Encrypted Virtualization \(SEV\) documentation](#) for more information.

6.2 AMD EPYC Virtualization Support

By default, the virtualization packages are not installed, as shown below. Be sure to install the virtualization packages of your choice, depending what you want to run, such as:

- A simple tool for developer platforms like [multipass](#)*.
- [LXD](#)* that does containers and virtualization.
- Direct use of [libvirt](#)* or [qemu](#)*.
- Small helpers like [uvtool](#)*.
- A potent openstack*.

Your selected virtualization packages will depend on the solution that you want to deploy.

For simplicity and CLI consumption, the following examples will use `qemu` and `libvirt` from the command line.

```
$ sudo apt install libvirt-daemon-system
```

Your system group setup may require you to log in again to pick up the membership to access virtualization functions through `libvirt`.

Validate that the virtualization host and packages are installed by executing the command `virt-host-validate` and then verifying that all of the validations show `PASS`. If not, then adjust the required parameters as recommended in the `virt-host-validate` output.

```
$ sudo virt-host-validate
QEMU: Checking for hardware virtualization : PASS
QEMU: Checking if device /dev/kvm exists : PASS
QEMU: Checking if device /dev/kvm is accessible : PASS
QEMU: Checking if device /dev/vhost-net exists : PASS
QEMU: Checking if device /dev/net/tun exists : PASS
QEMU: Checking for cgroup 'cpu' controller support : PASS
QEMU: Checking for cgroup 'cpuacct' controller support : PASS
QEMU: Checking for cgroup 'cpuset' controller support : PASS
```

```

QEMU: Checking for cgroup 'memory' controller support      : PASS
QEMU: Checking for cgroup 'devices' controller support    : PASS
QEMU: Checking for cgroup 'blkio' controller support      : PASS
QEMU: Checking for device assignment IOMMU support        : PASS
QEMU: Checking if IOMMU is enabled by kernel              : PASS
QEMU: Checking for secure guest support                   : PASS

```

The `qemu-system-$arch` command allows you to view and validate AMD processor support.

```

$ qemu-system-x86_64 -cpu help | grep -i amd
x86 EPYC-Milan-v1      AMD EPYC-Milan Processor
x86 EPYC-Rome-v1      AMD EPYC-Rome Processor
x86 EPYC-Rome-v2      AMD EPYC-Rome Processor
x86 EPYC-v1           AMD EPYC Processor
x86 EPYC-v2           AMD EPYC Processor (with IBPB)
x86 EPYC-v3           AMD EPYC Processor
x86 Opteron_G1-v1     AMD Opteron 240 (Gen 1 Class Opteron)
x86 Opteron_G2-v1     AMD Opteron 22xx (Gen 2 Class Opteron)
x86 Opteron_G3-v1     AMD Opteron 23xx (Gen 3 Class Opteron)
x86 Opteron_G4-v1     AMD Opteron 62xx class CPU
x86 Opteron_G5-v1     AMD Opteron 63xx class CPU
x86 phenom-v1         AMD Phenom(tm) 9550 Quad-Core Processor
3dnow 3dnowext 3dnowprefetch abm ace2 ace2-en acpi adx aes amd-no-ssb
amd-ssbd amd-stibp apic arat arch-capabilities avic avx avx-vnni avx2

```

The following command validates AMD EPYC support:

```

$ qemu-system-x86_64 -cpu help | grep -i epyc
x86 EPYC              (alias configured by machine type)
x86 EPYC-IBPB         (alias of EPYC-v2)
x86 EPYC-Milan        (alias configured by machine type)
x86 EPYC-Milan-v1     AMD EPYC-Milan Processor
x86 EPYC-Rome          (alias configured by machine type)
x86 EPYC-Rome-v1      AMD EPYC-Rome Processor
x86 EPYC-Rome-v2      AMD EPYC-Rome Processor
x86 EPYC-v1           AMD EPYC Processor
x86 EPYC-v2           AMD EPYC Processor (with IBPB)
x86 EPYC-v3           AMD EPYC Processor

```

6.3 Evaluating Workloads and VM Workloads

NUMA-aware or highly parallelizable workloads can take maximum advantage of AMD EPYC 9004 Series Processor architecture for performance tuning and gains. Memory-bound, NUMA friendly workloads can be parallelized to have each thread run independently. I/O-bound workloads can support multiple devices such that each device can remain connected to the original task owner. For example:

- The original STREAM benchmark can be parallelized with OpenMP and extended to measure NUMA-aware workload performance.
- The OpenMP and MPI Versions of the NASA Parallel Benchmarks (NPB) can be another good example test workload.

Both examples make very effective test VM workloads.

This page intentionally left blank.

Chapter

7

Troubleshooting and Debugging Notes

This section describes the following troubleshooting and debugging tools and procedures:

- [“Error Detection and Correction \(EDAC\)” on page 41](#)
- [“Error Injection using mce-inject” on page 43](#)

7.1 Error Detection and Correction (EDAC)

Ubuntu includes two AMD-compatible Error Detection and Correction (EDAC) modules for diagnosing memory errors:

Linux has two EDAC modules:

- `amd64_edac_mod` provides information and DRAM ECC-specific decoding. It loads automatically on supported systems if not blacklisted/disabled.
- `edac_mce_amd` provides more detailed MCA error decoding and is loaded by `amd64_edac_mod`.

```
# lsmod | grep -i edac
amd64_edac532480
edac_mce_amd368641 amd64 edac mod
```

System administrators monitoring ECC should monitor system health by continually recording both the number and rate of change of correctable and uncorrectable errors. The EDAC driver provides details about the number of memory controllers, memory controller characteristics, and physical DIMM characteristics (number of chip-select rows [`csrows`], and the channel tables).

The `/sys` filesystem for each `csrow` contain a number of entries with detailed information about the specific DIMM:

```
$ ls -ld /sys/devices/system/edac/mc/mc0/*
# correctable error count for this mem controller
/sys/devices/system/edac/mc/mc0/ce_count
/sys/devices/system/edac/mc/mc0/ce_noinfo_count
/sys/devices/system/edac/mc/mc0/max_location
/sys/devices/system/edac/mc/mc0/mc_name
/sys/devices/system/edac/mc/mc0/power
/sys/devices/system/edac/mc/mc0/rank0
/sys/devices/system/edac/mc/mc0/reset_counters
/sys/devices/system/edac/mc/mc0/sdram_scrub_rate
/sys/devices/system/edac/mc/mc0/seconds_since_reset
/sys/devices/system/edac/mc/mc0/size_mb
#un-correctable errors count for this mem controller
/sys/devices/system/edac/mc/mc0/ue_count
/sys/devices/system/edac/mc/mc0/ue_noinfo_count
/sys/devices/system/edac/mc/mc0/uevent
```

7.1.1 Get the Memory Controller MCx Device Information

EDAC driver messages help identify the DIMMs.

```
$ sudo dmesg | grep -i edac
[ 2.278438] EDAC MC: Ver: 3.0.0
[ 16.921113] EDAC amd64: MCT channel count: 1
[ 16.921200] EDAC MC0: Giving out device to module amd64_edac controller F19h_M10h: DEV
0000:00:18.3 (INTERRUPT)
[ 16.921204] EDAC amd64: F19h_M10h detected (node 0).
[ 16.921208] EDAC MC: UMC0 chip selects:
[ 16.921210] EDAC amd64: MC: 0: 32768MB 1: OMB
[ 16.921213] EDAC amd64: MC: 2: OMB 3: OMB
[ 16.921217] EDAC MC: UMC1 chip selects:
[ 16.921218] EDAC amd64: MC: 0: OMB 1: OMB
[ 16.921219] EDAC amd64: MC: 2: OMB 3: OMB
[ 16.921222] EDAC MC: UMC2 chip selects:
[ 16.921223] EDAC amd64: MC: 0: OMB 1: OMB
[ 16.921224] EDAC amd64: MC: 2: OMB 3: OMB
[ 16.921228] EDAC MC: UMC3 chip selects:
[ 16.921228] EDAC amd64: MC: 0: OMB 1: OMB
[ 16.921230] EDAC amd64: MC: 2: OMB 3: OMB
[ 16.921233] EDAC MC: UMC4 chip selects:
[ 16.921234] EDAC amd64: MC: 0: OMB 1: OMB
[ 16.921235] EDAC amd64: MC: 2: OMB 3: OMB
[ 16.921238] EDAC MC: UMC5 chip selects:
[ 16.921239] EDAC amd64: MC: 0: OMB 1: OMB
[ 16.921240] EDAC amd64: MC: 2: OMB 3: OMB
[ 16.921243] EDAC MC: UMC6 chip selects:
[ 16.921244] EDAC amd64: MC: 0: OMB 1: OMB
[ 16.921245] EDAC amd64: MC: 2: OMB 3: OMB
[ 16.921248] EDAC MC: UMC7 chip selects:
[ 16.921249] EDAC amd64: MC: 0: OMB 1: OMB
[ 16.921250] EDAC amd64: MC: 2: OMB 3: OMB
[ 16.921253] EDAC MC: UMC8 chip selects:
[ 16.921254] EDAC amd64: MC: 0: OMB 1: OMB
[ 16.921255] EDAC amd64: MC: 2: OMB 3: OMB
[ 16.921258] EDAC MC: UMC9 chip selects:
[ 16.921259] EDAC amd64: MC: 0: OMB 1: OMB
[ 16.921260] EDAC amd64: MC: 2: OMB 3: OMB
[ 16.921264] EDAC MC: UMC10 chip selects:
[ 16.921265] EDAC amd64: MC: 0: OMB 1: OMB
[ 16.921266] EDAC amd64: MC: 2: OMB 3: OMB
[ 16.921269] EDAC MC: UMC11 chip selects:
```

```
[ 16.921270] EDAC amd64: MC: 0:      OMB 1:      OMB
[ 16.921271] EDAC amd64: MC: 2:      OMB 3:      OMB
[ 16.921272] EDAC amd64: using x16 syndromes.
[ 16.921566] EDAC amd64: Node 1: No DIMMs detected.
[ 16.921579] EDAC PCI0: Giving out device to module amd64_edac controller EDAC PCI
controller: DEV 0000:00:18.0 (POLLED)
[ 16.921582] AMD64 EDAC driver v3.5.0
```

7.2 Error Injection using mce-inject

Machine checks are described in the [kernel](#)* as well as the [sysfs interfaces](#)* and the general topic of [hwpoisoning](#)*. [mce-inject](#)* is a common tool that provides valuable ECC diagnostics and debugging by injecting (simulating) machine check errors for testing on x86 architectures and is maintained by the kernel community.

7.2.1 Installation

To install `mce-inject`:

```
$ sudo apt install make gcc bison flex
$ git clone git://git.kernel.org/pub/scm/utils/cpu/mce/mce-inject.git
$ cd mce-inject
$ make
```

See [How to Install the mce-inject Tool](#)* for additional information.

7.2.2 Usage

To use `mce-inject`:

1. Load the kernel module.

```
$ sudo modprobe mce_inject
```

2. This comes with a sample file that simulates correctable errors. `mce-inject` uses these sample files to simulate the errors. Be aware that some more serious errors will make the machine stop or reboot by design.

```
# Run the tool
$ sudo ./mce-inject test/corrected

# Check the error occurring
$ sudo dmesg | tail -n 20
[1766233.705422] systemd[1]: Started Journal Service.
[1766233.793639] loop4: detected capacity change from 0 to 8
[1767004.106289] mce: Starting machine check poll CPU 0
[1767004.106303] mce: Machine check poll done on CPU 0
[1767004.106343] mce: [Hardware Error]: Machine check events logged
[1767004.106345] [Hardware Error]: Corrected error, no action required.
[1767004.106366] [Hardware Error]: CPU:0 (19:10:0) MC1_STATUS[|-|CE|-|AddrV|-|-|-|-|-]:
0x9400000000000000
[1767004.106390] [Hardware Error]: Error Addr: 0x0000000000000abcd
[1767004.106400] [Hardware Error]: IPID: 0x0000000000000000
[1767004.106402] mce: Starting machine check poll CPU 1
[1767004.106410] [Hardware Error]: Instruction Fetch Unit Ext. Error Code: 0, Op Cache
Microtag Probe Port Parity Error.
[1767004.106418] mce: Machine check poll done on CPU 1
[1767004.106428] [Hardware Error]: cache level: RESV, tx: INSN
[1767004.107584] mce: [Hardware Error]: Machine check events logged
[1767004.107585] [Hardware Error]: Corrected error, no action required.
```

```
[1767004.107596] [Hardware Error]: CPU:1 (19:10:0) MC2_STATUS[-|CE|-|AddrV|-|-|-|-|-]:  
0x9400000000000000  
[1767004.107613] [Hardware Error]: Error Addr: 0x00000000000001234  
[1767004.107624] [Hardware Error]: IPID: 0x0000000000000000  
[1767004.107633] [Hardware Error]: L2 Cache Ext. Error Code: 0, L2M Tag Multiple-Way-Hit  
error.  
[1767004.107648] [Hardware Error]: cache level: RESV, tx: INSN
```

Usually systems analyze logs or probe `mcelog`. If you require an immediate response, then you can register a trigger via the [sysfs interface](#)*

Chapter**8****Resources**

- [Memory Population Guidelines for AMD Family 19h Models 10h–1Fh](#) - Login required; please review the latest version if multiple versions are present.
- [Socket SP5 Platform NUMA Topology for AMD Family 19h Models 10h–1Fh](#) - Login required; please review the latest version if multiple versions are present.
- [Add support for Large Increment per Cycle Events](#)*
- [Ubuntu Server Docs](#)*
- [Community discussion at askubuntu.com](#)*
- [AMD Family 19h Models 00h–0Fh SP3 Power and Performance Optimization Guide](#) (login required)
- *High Performance Computing (HPC) Tuning Guide for AMD EPYC™ 9004 Series Processors* (available from [AMD EPYC Tuning Guides](#))

This page intentionally left blank.

Chapter

9

Processor Identification

Figure 9-1 shows the processor naming convention for AMD EPYC 9004 Series Processors and how to use this convention to identify particular processors models:

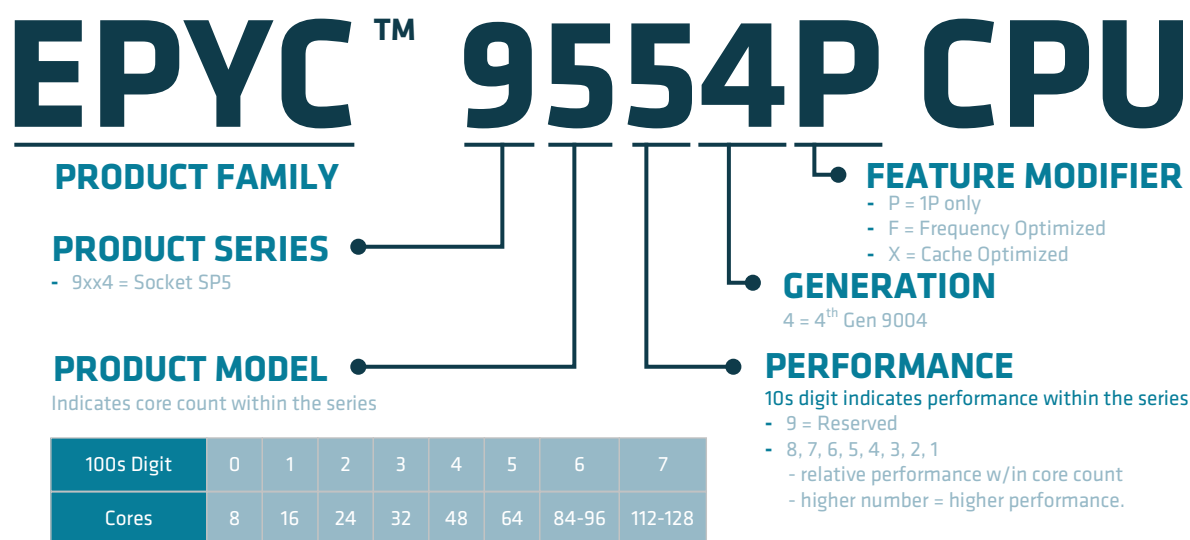


Figure 9-1: AMD EPYC SoC naming convention

9.1 CPUID Instruction

Software uses the **CPUID** instruction (`Fn0000_0001_EAX`) to identify the processor and will return the following values:

- **Family:** 19h identifies the “Zen 4” architecture
- **Model:** Varies with product. For example, EPYC Family 19h, Model 10h corresponds to an “A” part “Zen 4” CPU.
 - **91xx-96xx (including “X” OPNs):** Family 19h, Model 10-1F
 - **97xx:** Family 19h, Model A0-AF
- **Stepping:** May be used to further identify minor design changes

For example, **CPUID** values for Family, Model, and Stepping (decimal) of 25, 17, 1 correspond to a “B1” part “Zen 4” CPU.

9.2 New Software-Visible Features

AMD EPYC 9004 Series Processors introduce several new features that enhance performance, ISA updates, provide additional security features, and improve system reliability and availability. Some of the new features include:

- 5-level Paging
- AVX-512 instructions on a 256-byte datapath, including BFLOAT16 and VNNI support.
- Fast Short Rep STOSB and Rep CMPSB

Not all operating systems or hypervisors support all features. Please refer to your OS or hypervisor documentation for specific releases to identify support for these features.

Please also see the latest version of the [AMD64 Architecture Programmer's Manuals](#) or [Processor Programming Reference \(PPR\) for AMD Family 19h](#).

9.2.1 AVX-512

AVX-512 is a set of individual instructions supporting 512-bit register-width data (i.e., single instruction, multiple data [SIMD]) operations. AMD EPYC 9004 Series Processors implement AVX 512 by “double-pumping” 256-bit-wide registers. AMD's AVX-512 design uses the same 256-bit data path that exists throughout the Zen4 core and enables the two parts to execute on sequential clock cycles. This means that running AVX-512 instructions on AMD EPYC 9004 Series will cause neither drops on effective frequencies nor increased power consumption. On the contrary, many workloads run more energy-efficiently on AVX-512 than on AVX-256P.

Other AVX-512 support includes:

- Vectorized Neural Network Instruction (VNNI) instructions that are used in deep learning models and accelerate neural network inferences by providing hardware support for convolution operations.
- Brain Floating Point 16-bit (BFLOAT16) numeric format. This format is used in Machine Learning applications that require high performance but must also conserve memory and bandwidth. BFLOAT16 support doubles the number of SIMD operands over 32-bit single precision FP, allowing twice the amount of data to be processed using the same memory bandwidth. BFLOAT16 values mantissa dynamic range at the expense of one radix point.